Classification example: Adult Data Set

UCI Machine learning Repository에서 가져온 Adult dataset은 1994년 인구 조사 데이터베이스에서 베리 베커가 추출한 데이터로, 인구 정보를 바탕으로 연간 수입이 $50,000 이상인지의 여부를 예측하는 데이터이다. 데이터는 48842개의 행과 15개의 열로 이루어져 있다. 각 변수들에 대한 설명은 다음과 같다.

변수명	자료형	변수 설명
age	numeric	나이
workclass	character	직업 종류 (Private/Self-emp-not-inc/Local-gov/State-gov 등)
fnlwgt[1]	numeric	Census Bureau에서 할당한 가중치(비슷한 특성을 가질수록 비슷한 값을 가짐)
education	character	교육 수준(HS-grad/Some-college/Bachelors 등)
education-num	numeric	교육 수준에 대한 수치적 표현
marital-status	character	혼인 상태(Married-civ-spouse/Never-married 등)
occupation	character	직업(Tech-support/Sales 등)
relationship	character	가족 내 위치(Husband/Not-in-family 등)
race	character	인종(White/Black 등)
sex	character	성별(Male/Female)
capital-gain	numeric	자산 이익
capital-loss	numeric	자산 손실
hours-per-week	numeric	주간 근무시간
native-country	character	출신 국가(United-States/Mexico 등)
target	character	연간 수익 $50K 초과 여부(<=50K/>50K)

표 1 adult data variable description

표에서 위의 14개 열을 이용하여 해당하는 사람의 연간 수익이 $50K를 넘는지에 대해 예측한다. target 변수가 예측해야 할 변수이며, 설명변수는 6개의 numeric 변수와 8개의 character 변수로 이루어져 있다.

[1] 미국 인구조사국이 각 행에 할당한 가중치이다. 문자 그대로의 의미는 전체 데이터를 얻으려면 각 행을 final weight times(fnlwgt)만큼 복제해야 한다는 것을 말한다. 아마 61억 개의 행이 있을 텐데 이는 수십 년간 축적된 데이터이기 때문이다.

capital gain은 제외하고 분석을 진행하였다. [2]한편, f데이터의 첫 3행을 출력해보면 밑의 표와 같다.

age	workclass	fnlwgt	…	capital-loss	hours-per-week	native-country	target
0	39	State-gov	…	0	40	United-States	<=50K
1	50	Self-emp-not-inc	…	0	13	United-States	<=50K
2	38	Private	…	0	40	United-States	<=50K

표 2 adult data 첫 3 행 출력 예시

변수들 중 예측변수인 Target 변수의 분포는 다음과 같다.

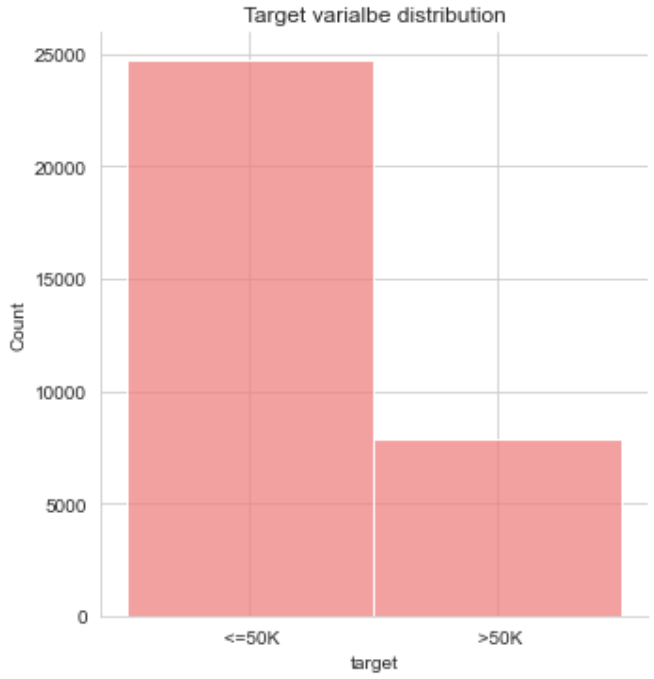


그림 1 소득 5 만 달러 이상, 5 만 달러 이하 비율

<=50K	>50K
22653개	7508 개
0.76	0.24

표 3 소득 5 만 달러 이상, 5 만 달러 이하 비율

연간 소득이 $50,000이하인 사람이 이상인 사람보다 많은 비율을 차지한다.

[2] *capital loss, capital gain 이게 소득에 영향을 미치는가? 소득이 여기에 영향을 미치는가? 애매하기 때문에 변수에서 제거하는 것이 맞다.

Numerical features의 변수별 통계량과 histogram은 다음과 같다. 수치형 변수들 중 capital-gain, capital-loss, fnlwgt 변수의 경우 0에 몰려 있는 자료임을 알 수 있다.

	mean	std	min	25%	50%	75%	max
age	38.58	13.64	17	28	37	48	90
fnlwgt	189978.37	105549.98	12285	117827	178356	237051	1484705
education-num	10.08	2.57	1	9	10	12	16
capital-gain	1077.65	7385.29	0	0	0	0	99999
capital-loss	87.30	402.96	0	0	0	0	4356
hours-per-week	40.44	12.35	1	40	40	45	99

표 4 numerical features의 통계량

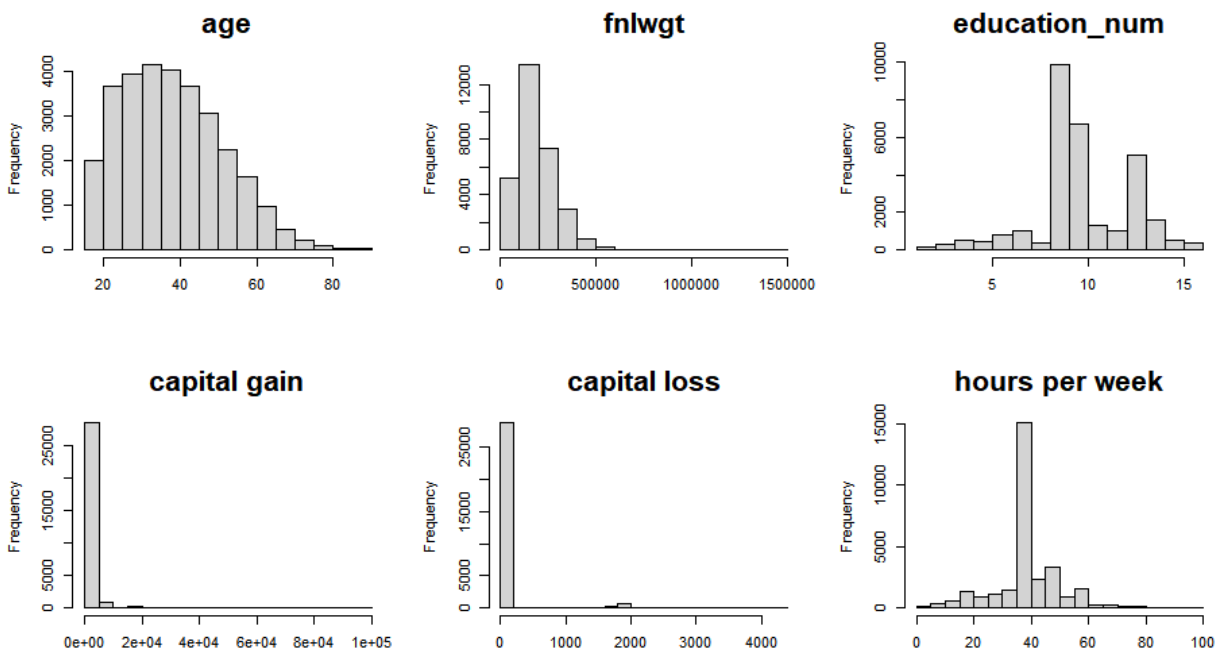


그림 2 numerical features의 변수별 분포 histogram

다음으로 범주형 변수들의 범주 별 히스토그램은 다음과 같다. Workclass의 경우 private(개인사업자), race는 white, Native-country의 경우 United states에 많은 데이터가 분포되어 있음을 알 수 있다.

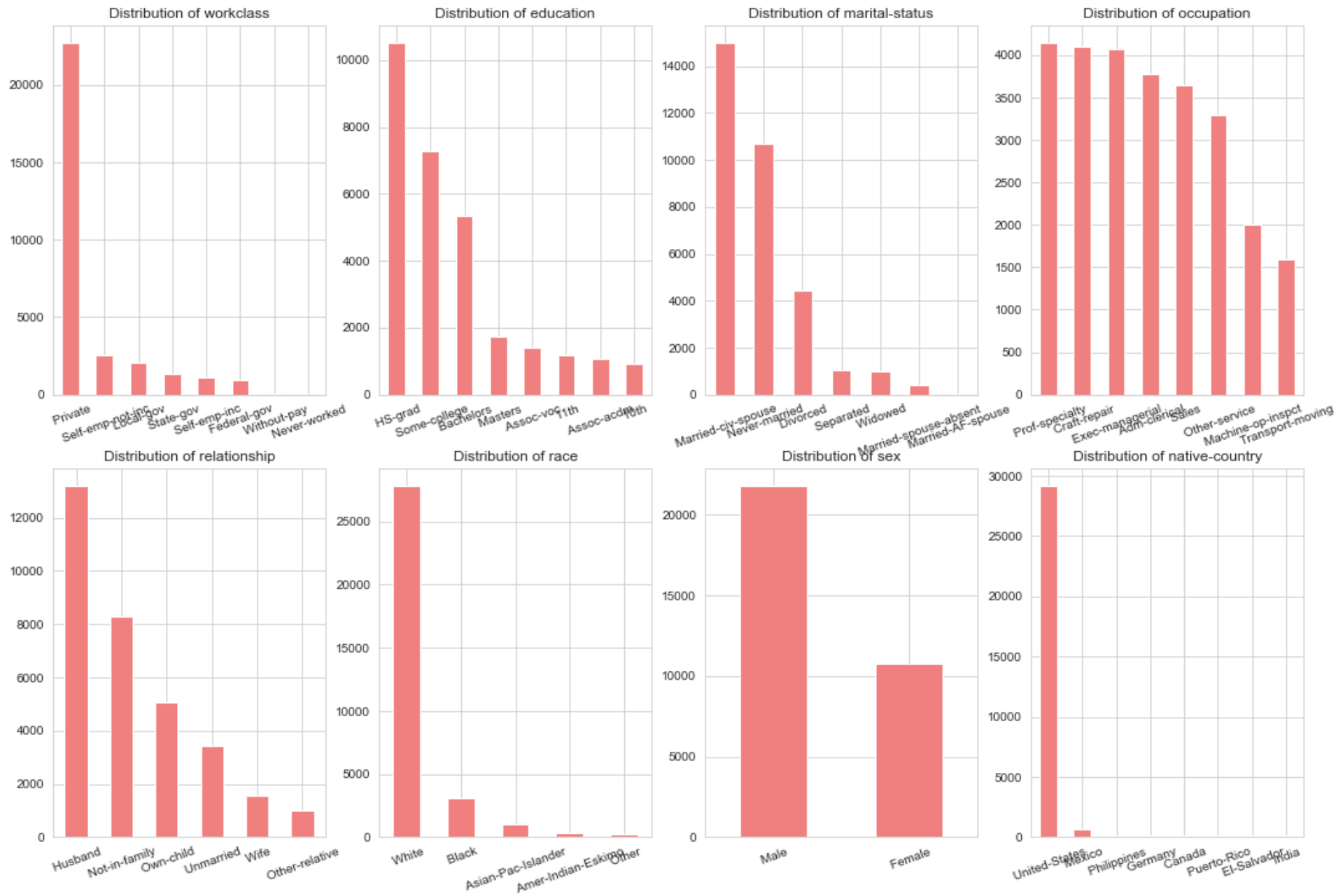


그림3 범주형 변수의 변수별 histogram

전체 데이터 개수 모두를 사용하여 분석하였으며, 데이터 셋 중 80%는 train set, 20%는 test set으로 사용하여 train set으로 모델을 적합하고 test set을 예측하여 에러를 계산한다. train data와 test data의 target 변수의 소득 5만 이상, 5만 이하 비율은 아래와 같다. 전체 데이터의 비율과 동일하다.

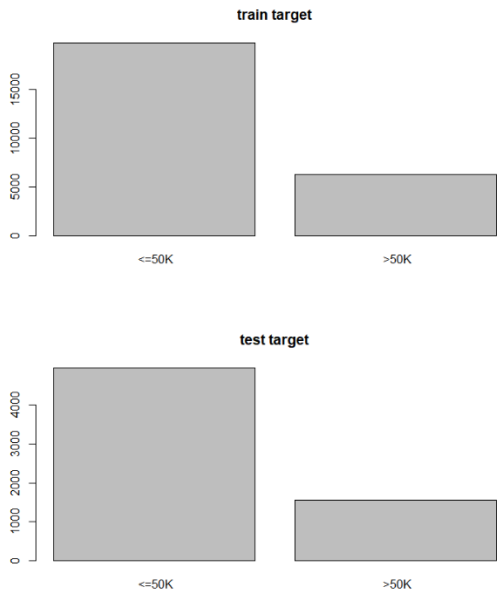
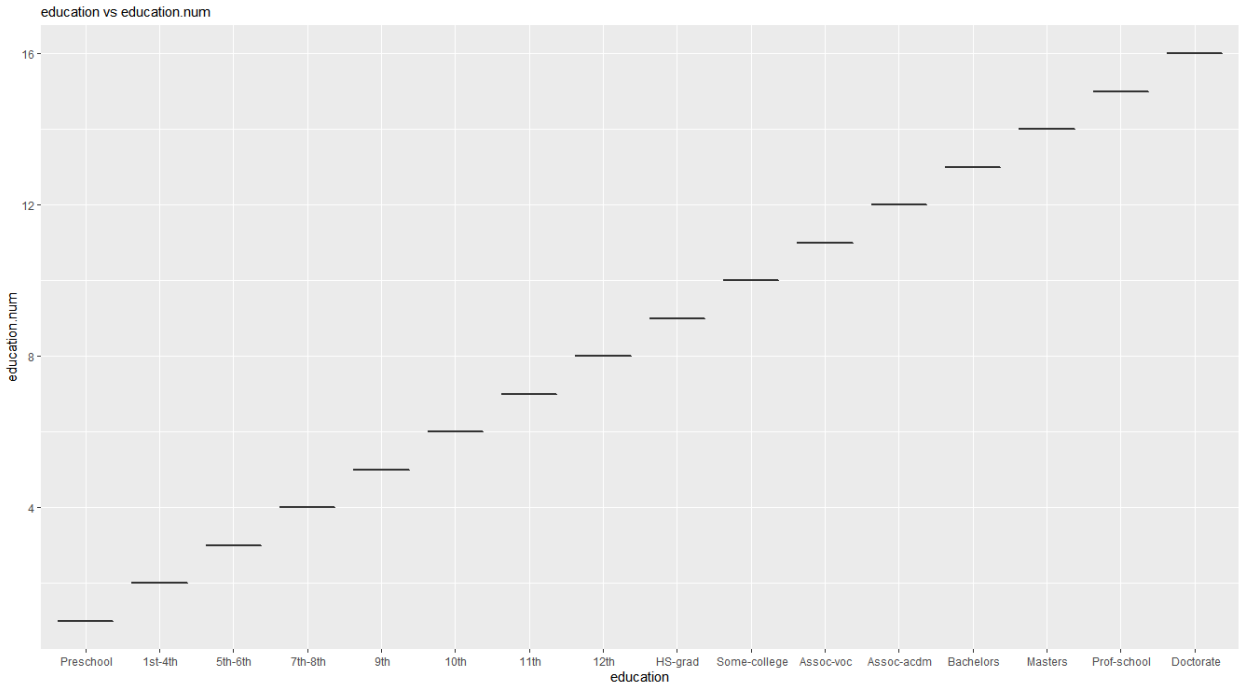


그림 4 train, test data target 변수 비율

	<=50K	>50K
train	18154개	5974개
target	0.75	0.25
test	4499개	1534개
target	0.75	0.25

표 5 train, test data target value count 및 비율

한편 education.num 과 education 간의 관계는 아래와 같다. 학력이 높을수록 education.num 은 더 증가한다.



XGboost

R과 Python 두 가지 언어로 모델을 실행하고 결과를 비교해본다. 두가지 케이스로 나눠 분석을 진행했다. 첫번째 케이스는 모든 parameter 값을 default로 두었다. 두번째 케이스는 learning rate를 작게 iteration을 크게 하여 모델을 적합하였다. 세번째 케이스는, grid search를 통해 가장 좋은 성능을 내는 파라미터 조합을 찾고 이를 이용하여 모델을 적합했다.

아래 표는 각 case별, 분석 툴 별 parameter values와 모델 학습 속도, 각 학습 모델에 대한 test 데이터의 예측 결과 및 feature importance plot이다. 첫번째 케이스가 두번째 케이스보다 정확도가 높았다. 한편 둘 다 분석 툴과 관계없이 동일한 결과가 나왔다. 세번째 케이스의 경우 그리드 서치를 통해 찾은 최적 파라미터 결과에 차이가 있었지만 예측 성능은 비슷했다. Accuracy 기준으로 세번째 케이스가 가장 예측 성능이 좋았다.

feature important plot은 파라미터가 동일한 경우 중요도 순서가 분석 툴에 관계없이 동일하게 나왔다. 케이스 3에서는 grid search를 통해 찾은 최적 파라미터 값이 달랐기 때문에 변수 중요도 순서에도 차이가 있었다. 첫번째 케이스에서는 센서스 가중치, 나이, 주당근무시간이 상위 3개의 중요변수이다. 두번째 케이스 에서는 나이, 교육수준, 자산 손실이 상위 3개의 중요변수이다. 세번째 케이스의 경우, R과 Python모두 가중치, 나이, 주당근무시간을 중요변수로 선택했다. 모든 경우에서 나이가 소득에 중요한 영향을 미치는 것을 알 수 있다.

아래 표는 각 케이스에 대한 모델학습속도, 평가지표에 대한 요약이며, 다음 페이지에는 보다 상세하게 케이스별 결과를 표로 나타냈다. 또한 마지막에는 feature importance plot을 비교하였다.

	모델학습속도		Accuracy		Recall	
	R	Python	R	Python	R	Python
case1	1.84	2.07	0.8465	0.8465	0.616	0.616
case2	7.33	8.57	0.8377	0.8377	0.5257	0.5257
case3	4.74	5.77	0.8508	0.85	0.6139	0.61

표 6 각 케이스 결과에 대한 요약

각 케이스의 parameter 값 과 분석 툴 별 system time

	parameter	values	Optimal parameter		grid search 걸린 시간		모델 학습 속도(second)	
			R	Python	R	Python	R	Python
case1	learning_rate	0.3(Default)					1.84	2.079
	iterations	100(Default)						
case2	max_depth	2					7.33	8.57
	learning rate	0.01						
	iterations	1000						
case3 grid search	max_depth	[2,4,6]	4	4	약 2시간 31분	약 1시간 50분	4.74	5.77
	learning_rate(alias=eta)	[0.05,0.1]	0.05	0.05				
	gamma	[0.1, 0.5, 1]	0.5	0.1				
	min_child_weight	[0.5, 1]	0.5	0.5				
	colsample_bytree	[0.6,0.8]	0.6	0.8				
	iterations	[500,1000]	500	500				

R로 분석한 결과

R에서 case1으로 학습한 모델의 test data 예측 결과

case1		실제값		평가지표	값
		0	1	Accuracy	0.8465
예측값	0	4219	552	Recall	0.616
	1	374	888	Precision	0.70

R에서 case2으로 학습한 모델의 test data 예측 결과

case1		실제값		평가지표	값
		0	1	Accuracy	0.8377
예측값	0	4297	683	Recall	0.5257
	1	296	757	Precision	0.7189

R에서 case3의 최적 파라미터로 학습한 모델의 test data 예측 결과

case1		실제값		평가지표	값
		0	1	Accuracy	0.8508
예측값	0	4249	556	Recall	0.6139
	1	344	884	Precision	0.7199

Python으로 분석한 결과

Python에서 case1으로 학습한 모델의 test data 예측 결과

case2		실제값		평가지표	값
		0	1	Accuracy	0.8465
예측값	0	4219	552	Recall	0.616
	1	374	888	Precision	0.70

Python에서 case2으로 학습한 모델의 test data 예측 결과

case1		실제값		평가지표	값
		0	1	Accuracy	0.8377
예측값	0	4297	683	Recall	0.5257
	1	296	757	Precision	0.7189

Python에서 case3의 최적 파라미터로 학습한 모델의 test data 예측 결과

case1		실제값		평가지표	값
		0	1	Accuracy	0.85
예측값	0	4248	563	Recall	0.61
	1	345	877	Precision	0.72

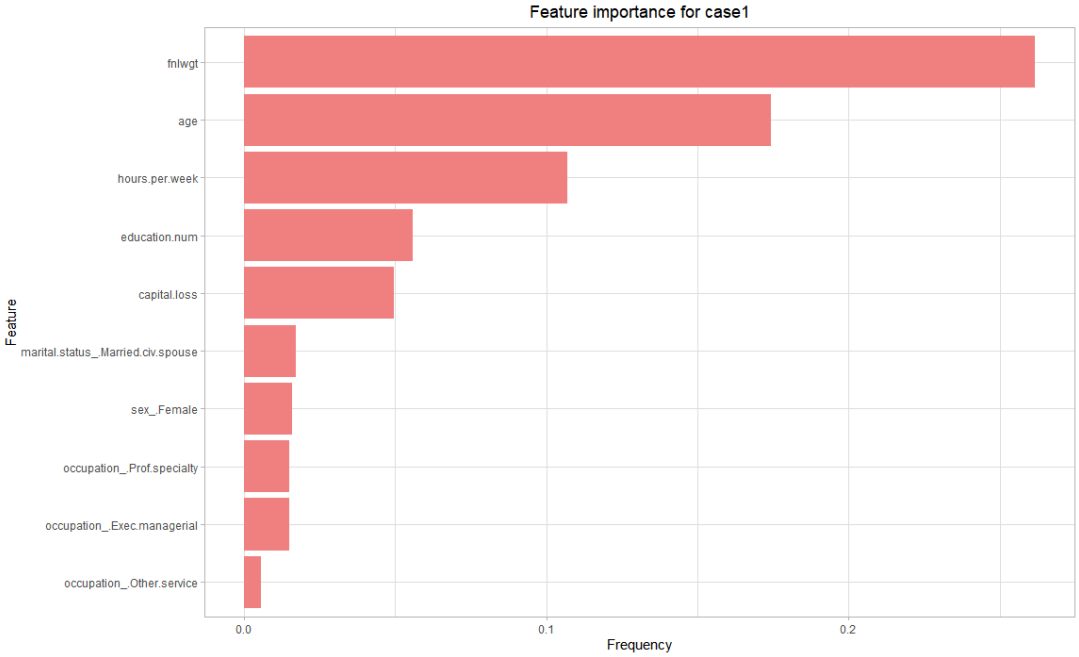


그림 6 R로 case1을 학습한 모델에서의 feature importance plot

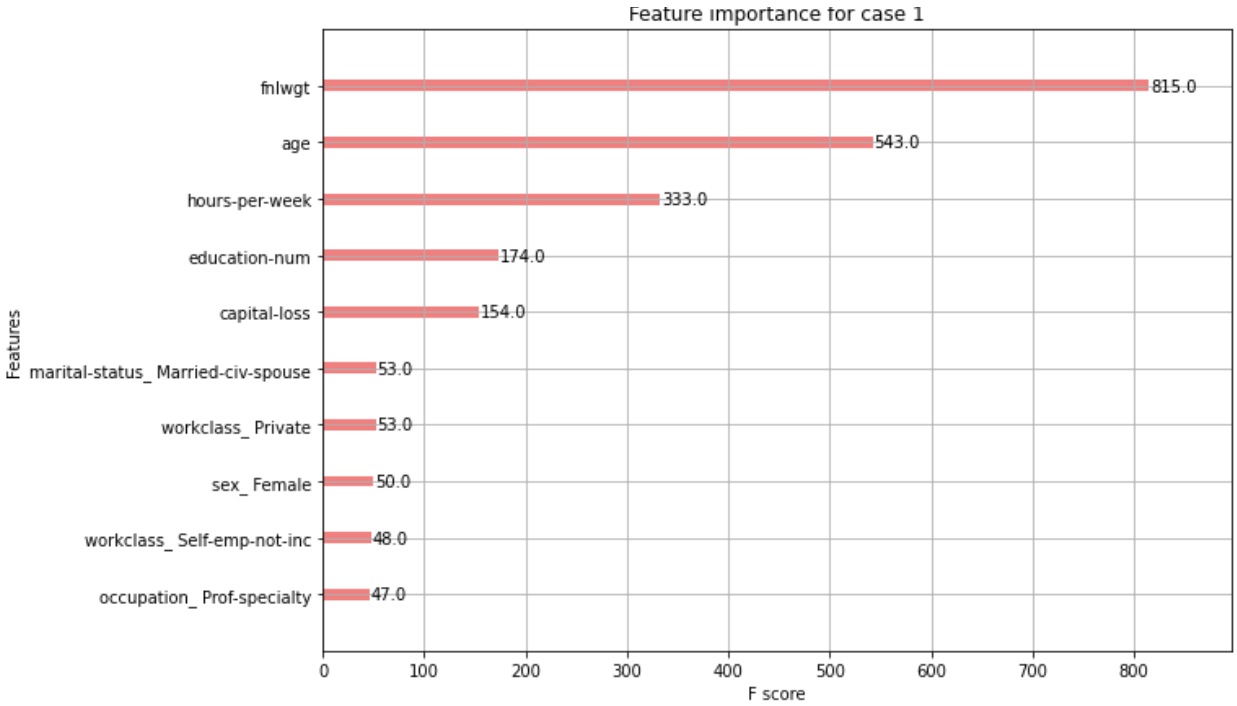


그림 5 Python으로 case1을 학습한 모델에서의 feature importance plot

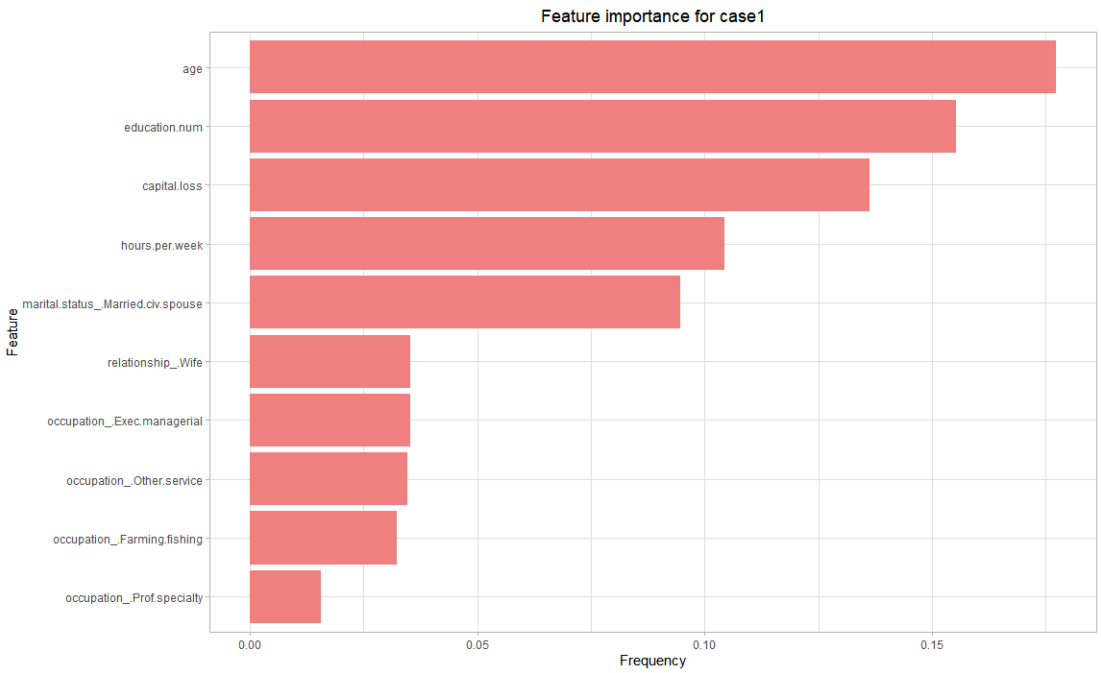


그림 8 R로 case2를 학습한 모델에서의 feature importance plot

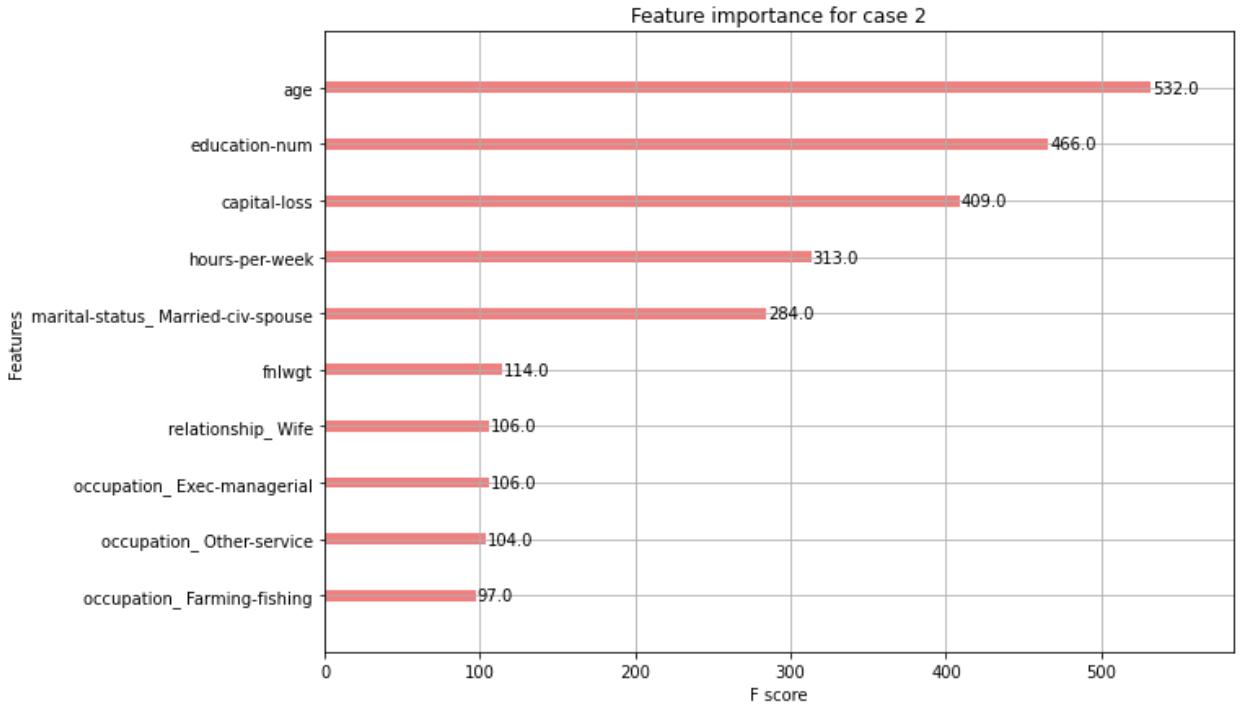


그림 7 Python으로 case2를 학습한 모델에서의 feature importance plot

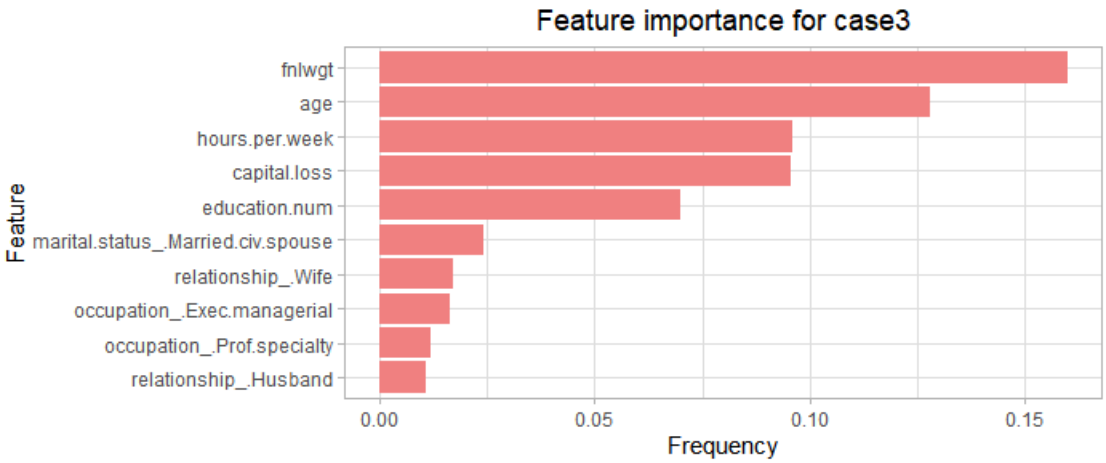


그림 9 R 에서 case 3 의 최적 파라미터로 학습한 모델에서의 feature importance plot

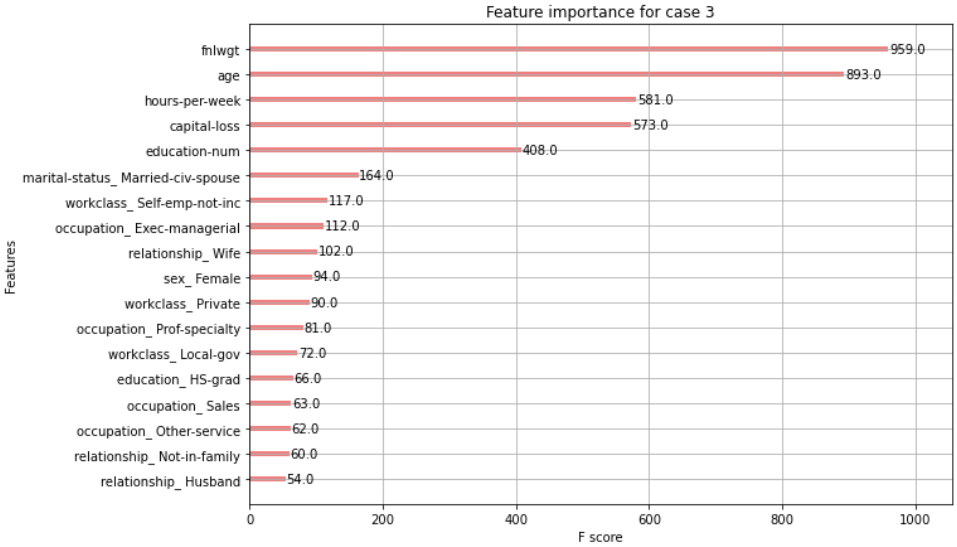


그림 10 Python 에서 case 3 의 최적 파라미터로 학습한 모델에서의 feature importance plot

Partial dependence plot 은 각 케이스마다 Feature importance plot 의 상위 3 개 변수에 대하여 그렸다.

Case1 에 대한 PDP 를 보면, 나이가 들수록 소득이 증가하다가 50 살을 기점으로 소득이 감소하는 추세를 보인다. 교육수준은 증가할수록 소득이 가파르게 증가한다. 주당 근무시간의 경우 초반에는 소득이 천천히 증가하다가 근무시간이 길어질수록 소득이 빠르게 증가하며 일정 수준을 넘어가면 소득 증가에 큰 영향을 미치지 못한다.

Case2 에 대한 PDP 를 보면, 나이가 증가할수록 소득이 오르다가 일정 수준이 지나면 소득에 영향이 없다. 교육수준은 증가할수록 소득이 가파르게 증가한다. 자본 손실은 일정한 패턴을 보이지 않는다.

Case3 에 대한 PDP 를 보면, fnlwgt 는 Cenus 에서 할당한 가중치로서, 대체로 가중치가 클수록 소득이 증가한다. 나이가 들수록 소득이 증가하다가 50 살을 기점으로 소득이 감소하며 나이가 너무 많으면 소득이 급격히 줄어든다. 주당 근무시간은 많이 일할수록 소득이 증가하지만 일정 수준을 넘어가면 소득 증가에 큰 영향을 미치지 못한다.

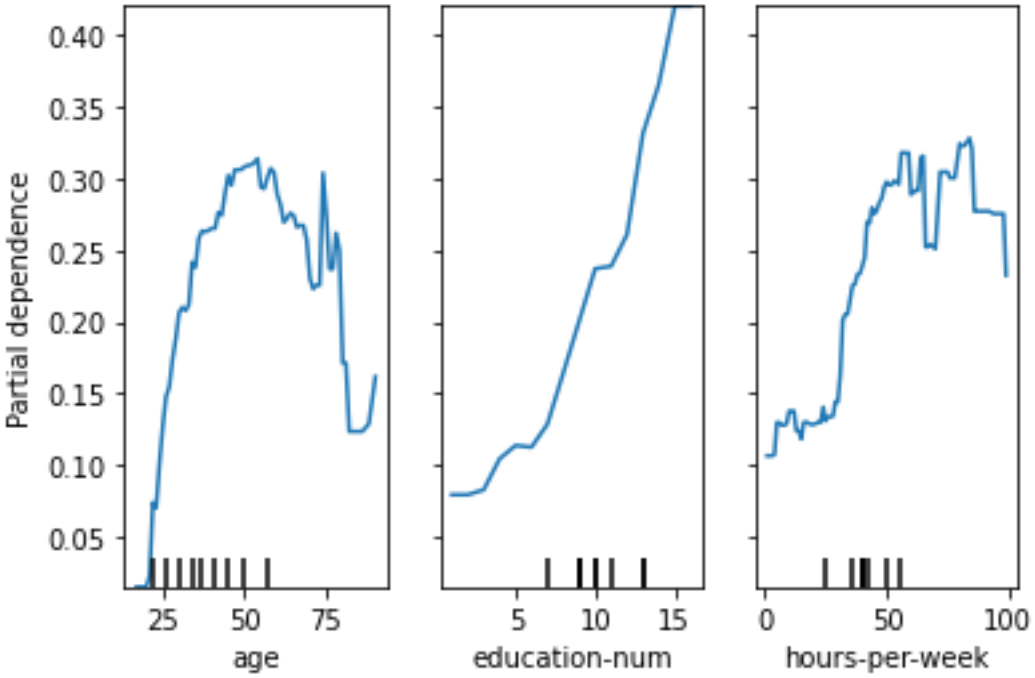


그림 11 case1 에 대한 partial dependence plot

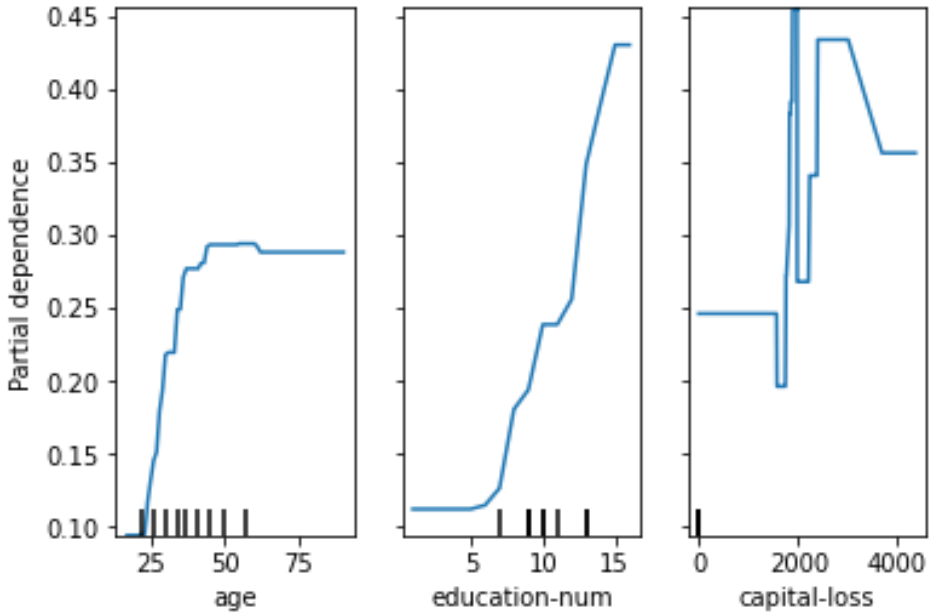


그림 12 case2 에 대한 partial dependence plot

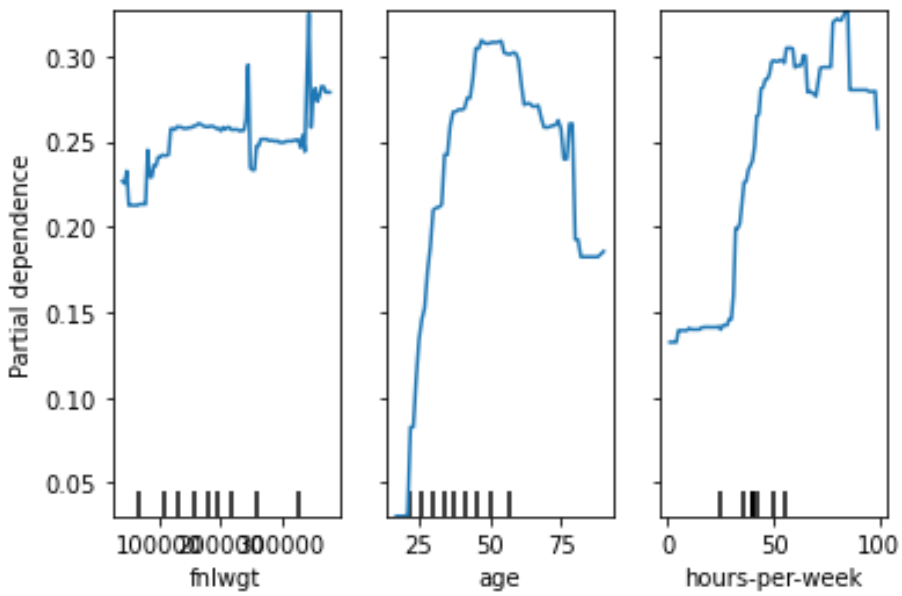


그림 13 case3 에 대한 partial dependence plot

[3]

[3] *PDP 그래프가 위로 솟는 이유는 데이터가 없어서 평균 냈을 때 값이 줄어들지 않아서이다.

Catboost

R과 Python 두 가지 언어로 모델을 실행하고 결과를 비교해본다. 여섯 가지 케이스로 나눠 분석을 진행했다. 먼저 boosting 방법이 ordered 일 때와 Plain일 때를 나눴다. 다음으로 파라미터 조정을 위해 세가지 케이스를 만들었다. 첫번째 케이스는 default값으로 모델을 적합하였다. 이때 learning rate는 catboost 함수 내에서 자동으로 설정되었다. 두번째 케이스는 max depth와 learning rate는 작게, n estimator는 크게 하여 모델을 적합했다. case3은 grid search를 통해 가장 좋은 성능을 내는 파라미터 조합을 찾고 이를 이용하여 모델을 적합했다.

방법I : Ordered boosting + case1
방법II : Ordered boosting + case2
방법III : Ordered boosting + case3
방법IV : Plain boosting + case1
방법V : Plain boosting + case2
방법VI: Plain boosting + case3

[참고: 6가지 방법에 대한 요약]

아래 표는 각 케이스에 대한 모델학습속도, 평가지표에 대한 요약이다. Ordered Boosting과 Plain의 성능차이는 크지 않았다. 모델 학습 속도는 default값일 때 더 시간이 오래 걸렸다. 또한 R이 파이썬 보다 1-2초정도 빨랐다. 이는 default값으로 모델을 학습시키게 되면 모델 내에서 파라미터를 최적화하기 때문에 시간이 더 오래 걸리는 것으로 보인다. 또한 case1이 case2보다 예측성능이 좋았다. 이는 Catboost함수의 default 파라미터가 기본적으로 최적화가 잘 되어있기 때문이다. 그리드 서치를 거친 case3보다 디폴드 값으로 돌린 case1의 예측 성능이 가장 좋다.

Feature importance plot의 경우, 각 케이스마다 차이가 있었지만 공통적으로 age를 모두 중요 변수로 꼽았다. 또한 두번째로 가장 많이 나온 변수는 relationship이다. 이외에는 자산 손실, 주당근무시간, 결혼상태가 중요 변수로 선택되었다.

Parital dependence plot의 경우, 피처별로 비슷한 그래프가 그려졌다. 나이는 증가할수록 소득이 증가하나 약 60세 이후로 소득이 줄어들었다. 자산 손실이 소득에 미치는 영향은 뚜렷하지 않았다. 교육수준은 증가할 수록 소득이 빠르게 증가한다.

다음 페이지에는 보다 상세하게 케이스별 결과를 표로 나타냈다. 또한 마지막에는 feature importance plot과 partial dependence plot을 비교하였다.

		모델학습속도		Accuracy		Recall	
		R	Python	R	Python	R	Python
case1	Ordered	60.01	62.64	0.851	0.85	0.6118	0.62
	Plain	51.54	54.45	0.8495	0.85	0.6208	0.62
case2	Ordered	18.47	19.72	0.8391	0.84	0.5653	0.57
	Plain	15.90	15.33	0.8414	0.84	0.5653	0.57
case3	Ordered	7.537	26.38	0.8444	0.85	0.5868	0.62
	Plain	6.82	20.96	0.8449	0.85	0.5938	0.61

표 7 각 케이스 결과에 대한 요약

Ordered Boosting에서 각 케이스별 parameter 값 과 system time

	parameter	values	Optimal parameter		grid search 걸린 시간		모델 학습 속도(second)	
			R	Python	R	Python	R	Python
case1	learning rate	iteration에 따라 자동 설정	0.108997	0.069565			60.01	62.64
	iterations	1000						
case2	max_depth	2					18.47	19.72
	learning rate	0.01						
	iterations	1000						
case3 grid search	max_depth	[2,4,6]	2	6	1시간 9분	1시간 5초	7.537	26.38
	learning_rate	[0.05,0.1]	0.05	0.1				
	l2_leaf_reg	[0.1, 0.5, 1]	0.1	1				
	rsm	[0.5, 0.85]	0.5	0.85				
	border count	[64,254]	64	64				
	iterations	[500,1000]	500	500				

Plain Boosting에서 각 케이스별 parameter 값 과 system time

	parameter	values	Optimal parameter		grid search 걸린 시간		모델 학습 속도(second)	
			R	Python	R	Python	R	Python
case1	learning rate	iteration에 따라 자동 설정	0.108997	0.069565			51.54	54.45
	iterations	1000						
case2	max_depth	2					15.90	15.33
	learning rate	0.01						
	iterations	1000						
case3 grid search	max_depth	[2,4,6]	2	6	1시간36초	46분 52초	6.82	20.96
	learning_rate	[0.05,0.1]	0.05	0.1				
	l2_leaf_reg	[0.1, 0.5, 1]	0.1	1				
	rsm	[0.5, 0.85]	0.5	0.85				
	border count	[64,254]	64	64				
	iterations	[500,1000]	500	500				

Ordered Boosting

R, Ordered Boosting, Case1일때, 학습한 모델의 test data 예측 결과

case1		실제값		평가지표	값
		0	1	Accuracy	0.851
예측값	0	4253	559	Recall	0.6118
	1	340	881	Precision	0.7215

Python, Ordered Boosting, Case1일때, 학습한 모델의 test data 예측 결과

case1		실제값		평가지표	값
		0	1	Accuracy	0.85
예측값	0	4235	548	Recall	0.62
	1	358	892	Precision	0.71

R, Ordered Boosting, Case2일때, 최적 파라미터로 학습한 모델의 예측 결과

case1		실제값		평가지표	값
		0	1	Accuracy	0.8391
예측값	0	4248	626	Recall	0.5653
	1	345	814	Precision	0.7023

Python, Ordered Boosting, Case2일때, 최적 파라미터로 학습한 모델의 예측 결과

case1		실제값		평가지표	값
		0	1	Accuracy	0.84
예측값	0	4253	623	Recall	0.57
	1	340	817	Precision	0.71

Plain Boosting

R, Plain Boosting, Case1일때, 학습한 모델의 test data 예측 결과

case1		실제값		평가지표	값
		0	1	Accuracy	0.8495
예측값	0	4231	546	Recall	0.6208
	1	362	894	Precision	0.7118

Python, Plain Boosting, Case1일때, 학습한 모델의 test data 예측 결과

case1		실제값		평가지표	값
		0	1	Accuracy	0.85
예측값	0	4246	544	Recall	0.62
	1	347	896	Precision	0.72

Python, Plain Boosting, Case2일때, 최적 파라미터로 학습한 모델의 예측 결과

case1		실제값		평가지표	값
		0	1	Accuracy	0.8414
예측값	0	4262	626	Recall	0.5653
	1	331	814	Precision	0.7109

Python, Plain Boosting, Case2일때, 최적 파라미터로 학습한 모델의 예측 결과

case1		실제값		평가지표	값
		0	1	Accuracy	0.84
예측값	0	4264	618	Recall	0.57
	1	329	822	Precision	0.71

R, Ordered Boosting, Case3일때, 최적 파라미터로 학습한 모델의 예측 결과

case1		실제값		평가지표	값
		0	1	Accuracy	0.8444
예측값	0	4249	595	Recall	0.5868
	1	344	845	Precision	0.7107

Python, Ordered Boosting, Case3일때, 최적 파라미터로 학습한 모델의 예측 결과

case1		실제값		평가지표	값
		0	1	Accuracy	0.85
예측값	0	4232	547	Recall	0.62
	1	361	893	Precision	0.71

R, Plain Boosting, Case3일때, 최적 파라미터로 학습한 모델의 예측 결과

case1		실제값		평가지표	값
		0	1	Accuracy	0.8449
예측값	0	4242	585	Recall	0.5938
	1	351	855	Precision	0.7090

Python, Plain Boosting, Case3일때, 최적 파라미터로 학습한 모델의 예측 결과

case1		실제값		평가지표	값
		0	1	Accuracy	0.85
예측값	0	4225	558	Recall	0.61
	1	368	882	Precision	0.71

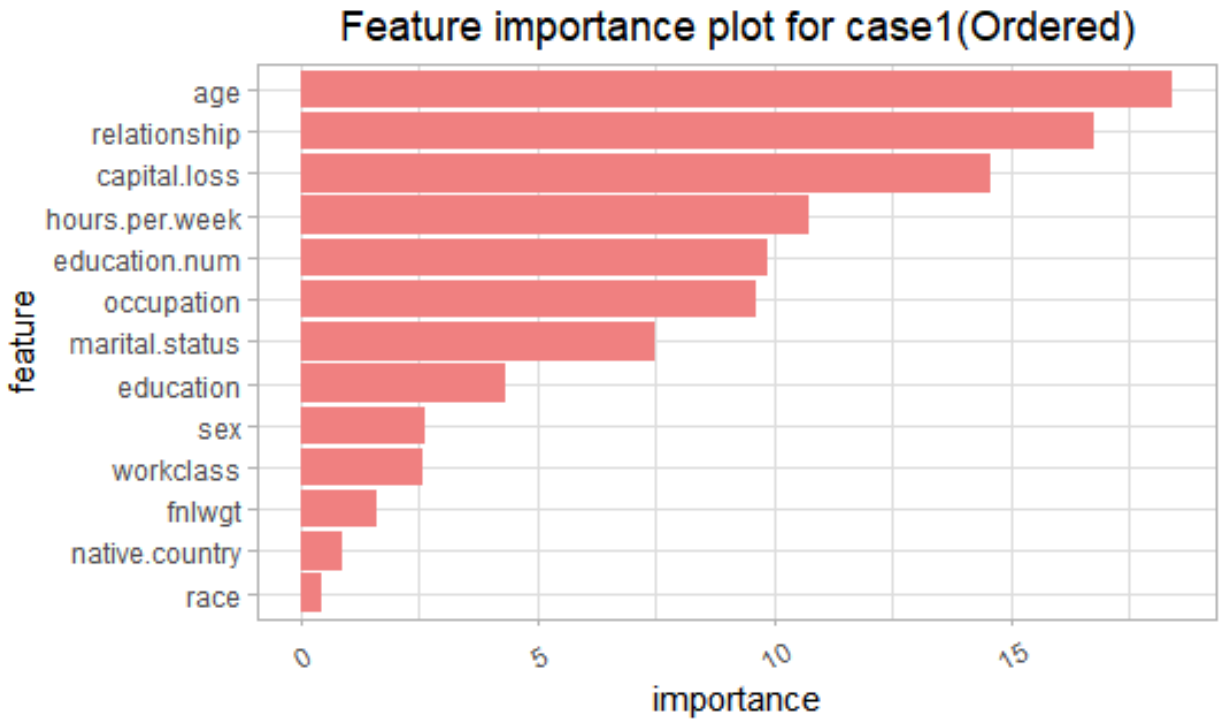


그림 14 R로 Ordered Boosting 이면서 case1 일 때 학습한 모델에서의 feature importance plot

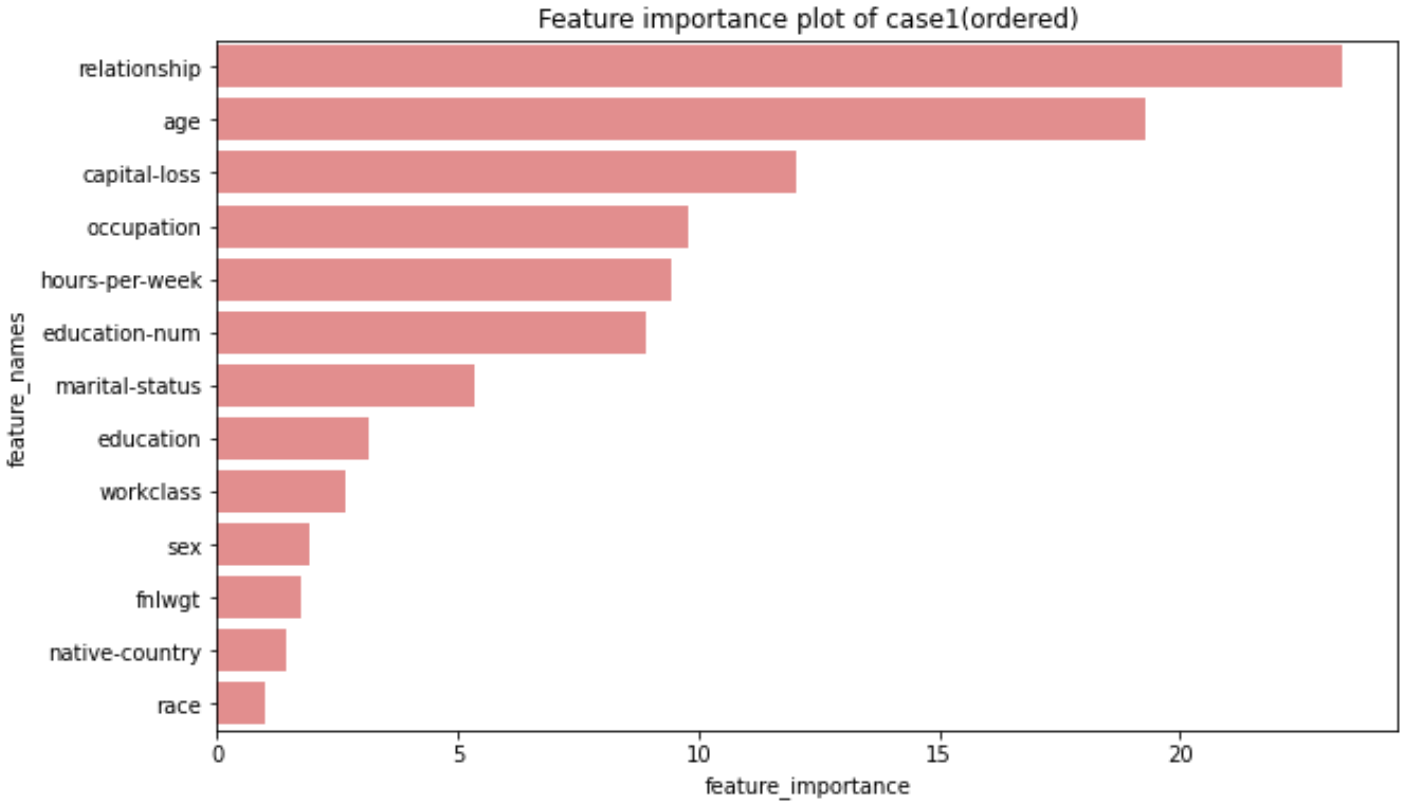


그림 15 Python으로 Ordered Boosting 이면서 case1 일 때 학습한 모델에서의 feature importance plot

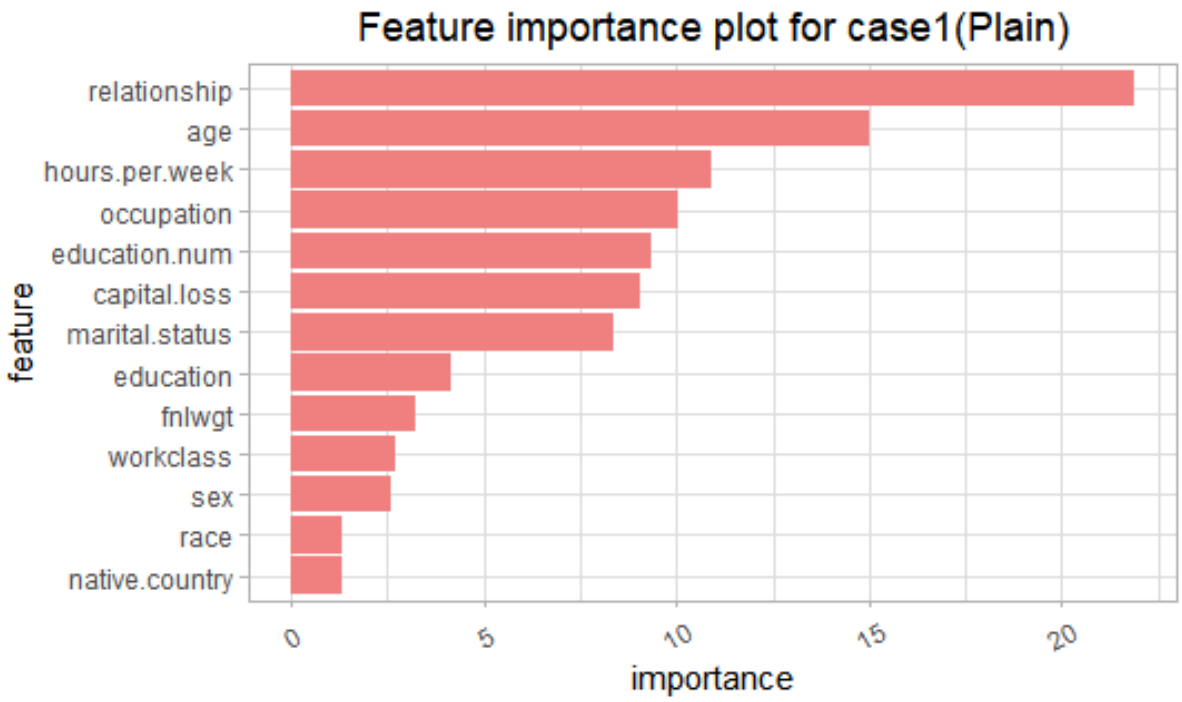


그림 16 R로 Plain Boosting 이면서 case1 일 때 학습한 모델에서의 feature importance plot

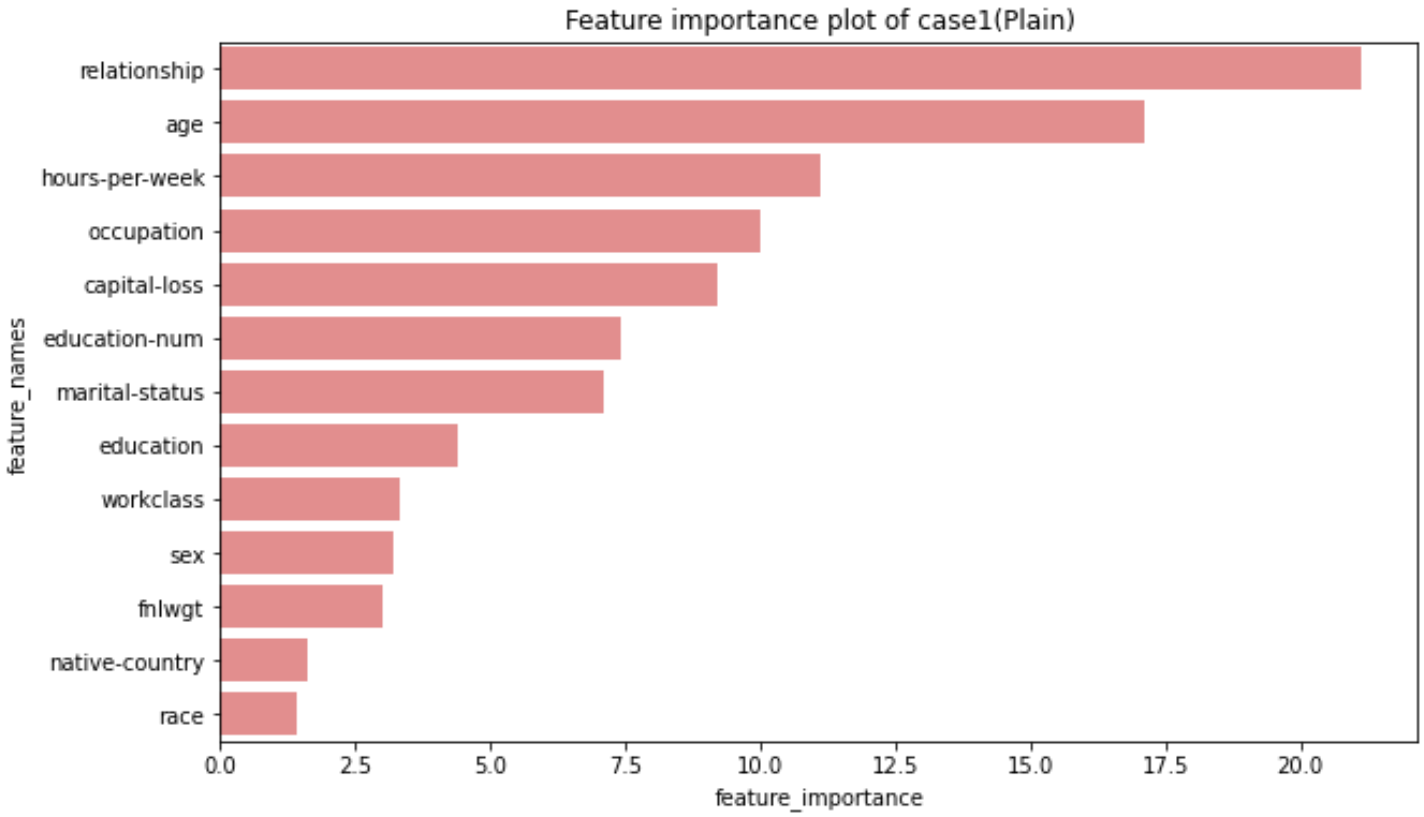


그림 17 Python으로 Plain Boosting 이면서 case1 일 때 학습한 모델에서의 feature importance plot

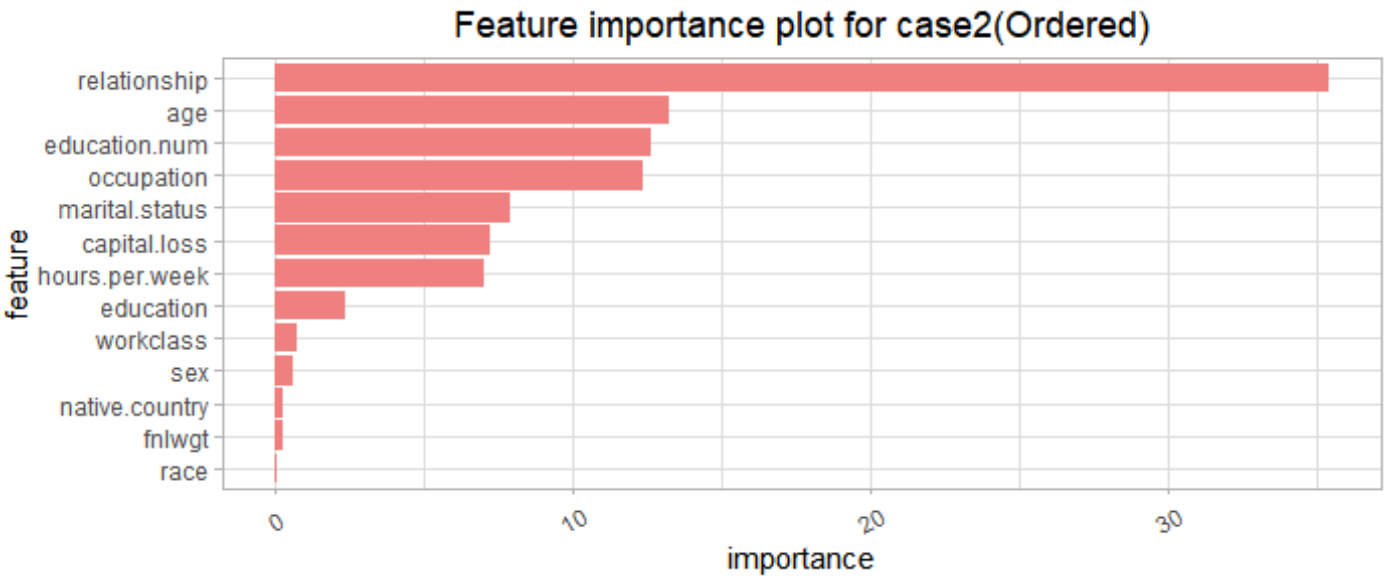


그림 18 R로 ordered Boosting 이면서 case2 일 때 학습한 모델에서의 feature importance plot

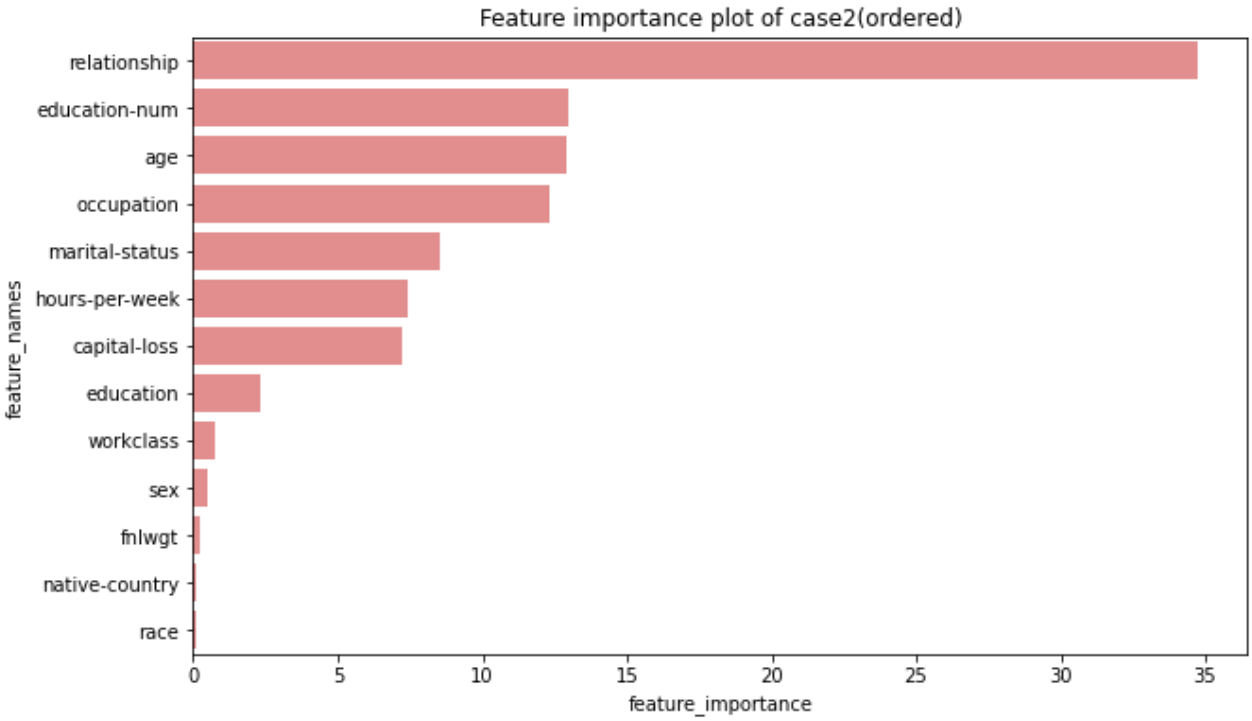


그림 19 Python으로 Ordered Boosting 이면서 case2 일 때 학습한 모델에서의 feature importance plot

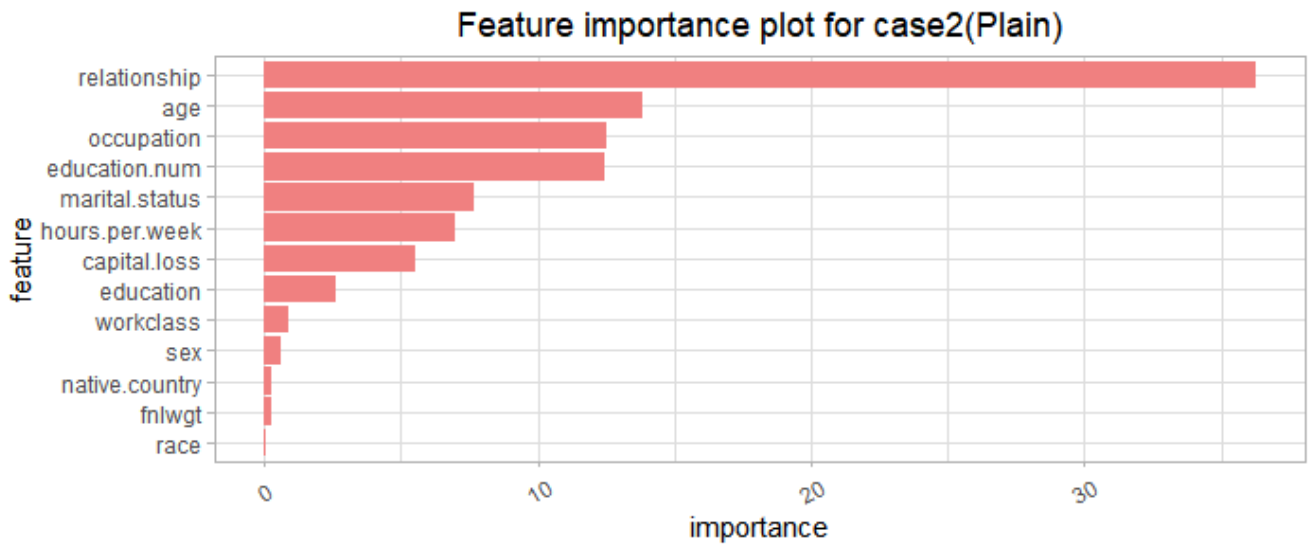


그림 20 R로 Plain Boosting 이면서 case2 일 때 학습한 모델에서의 feature importance plot

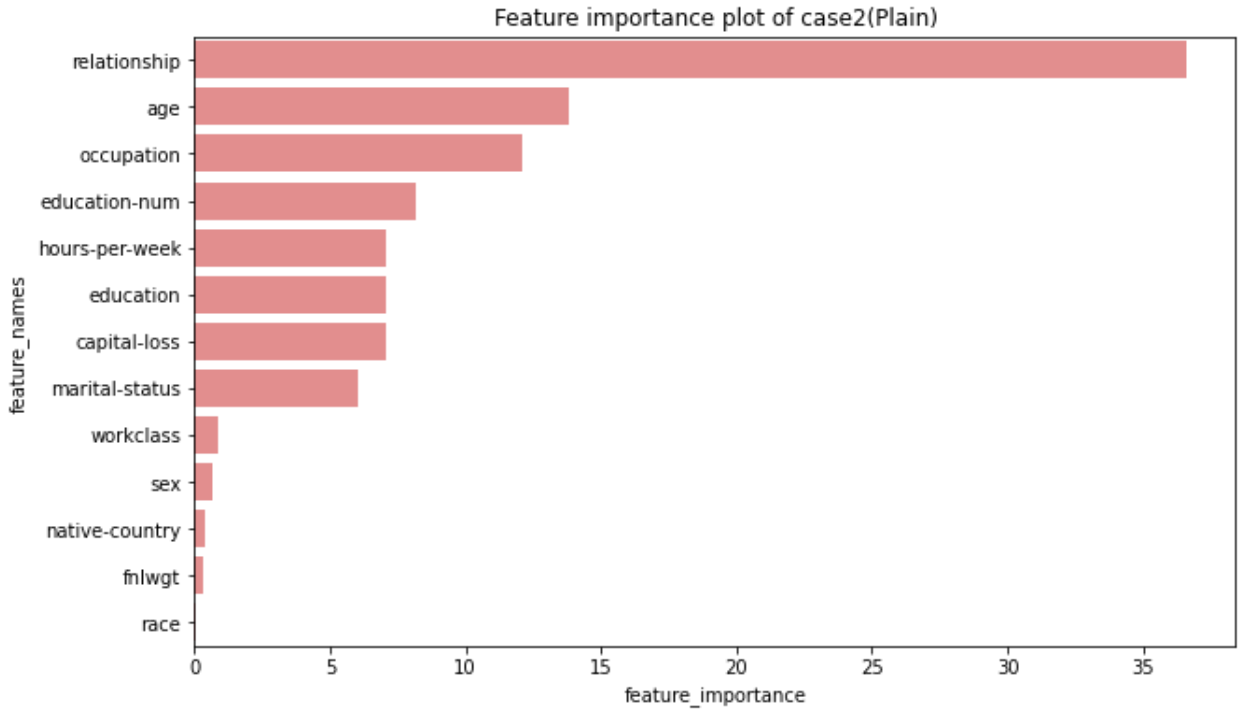


그림 21 Python으로 Plain Boosting 이면서 case2 일 때 학습한 모델에서의 feature importance plot

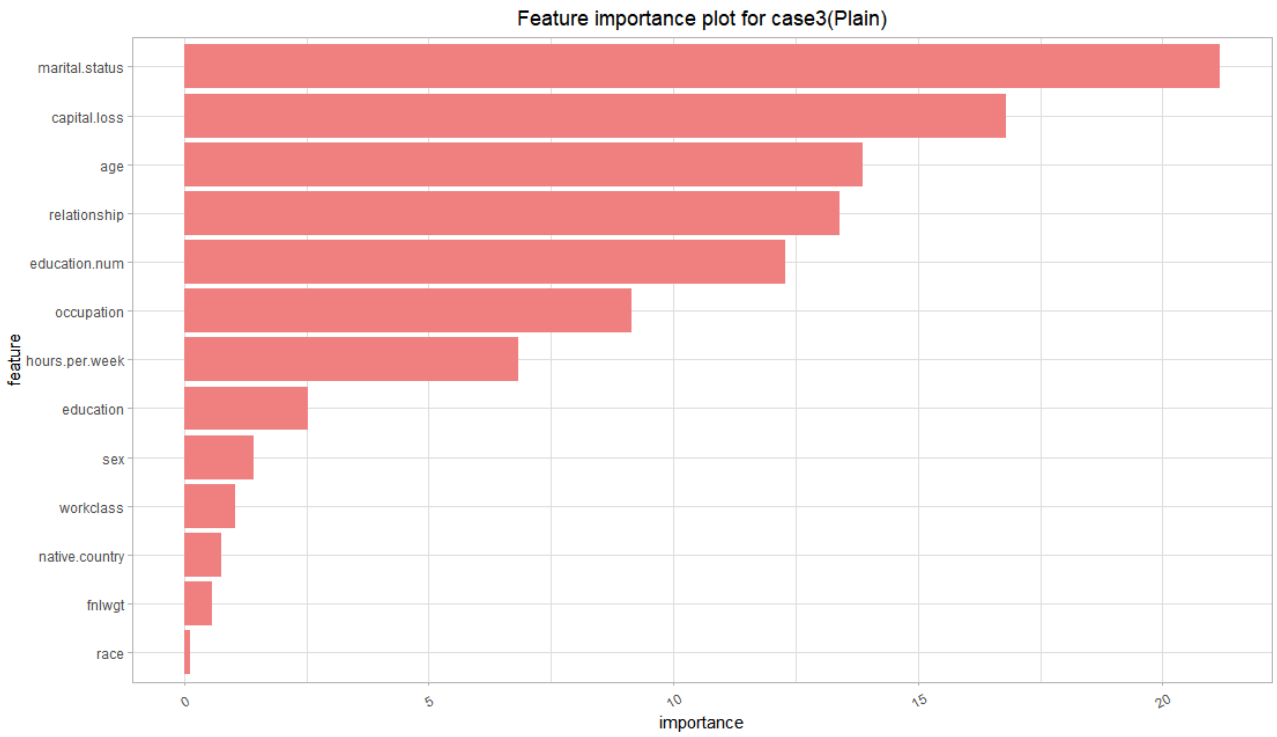


그림 22 R로 Plain Boosting 이면서 case3 일 때 학습한 모델에서의 feature importance plot

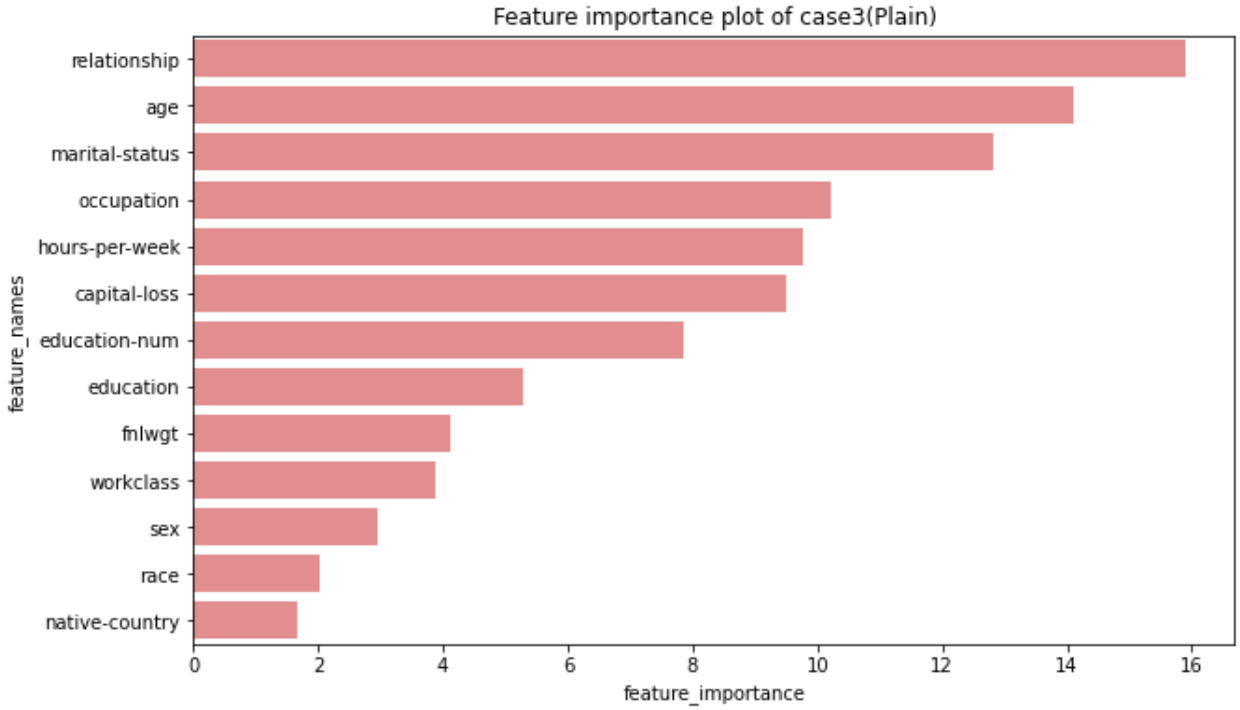


그림 23 Python으로 Plain Boosting 이면서 case3 일 때 학습한 모델에서의 feature importance plot

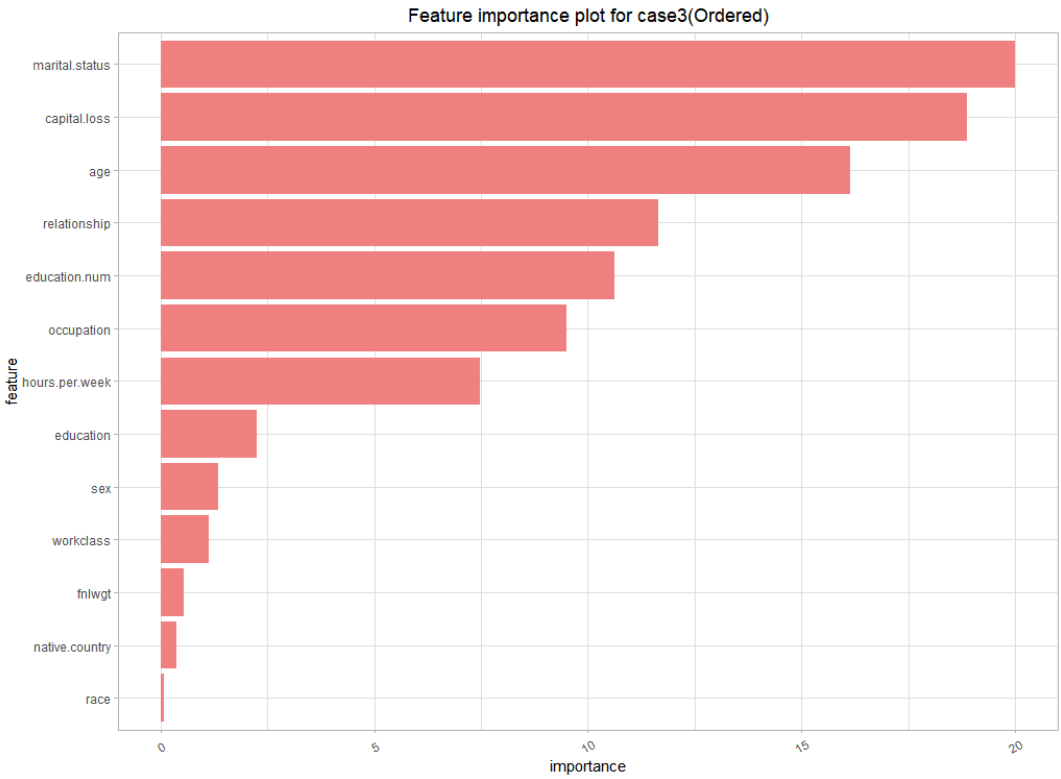


그림 24 R로 ordered Boosting 이면서 case3 일 때 학습한 모델에서의 feature importance plot

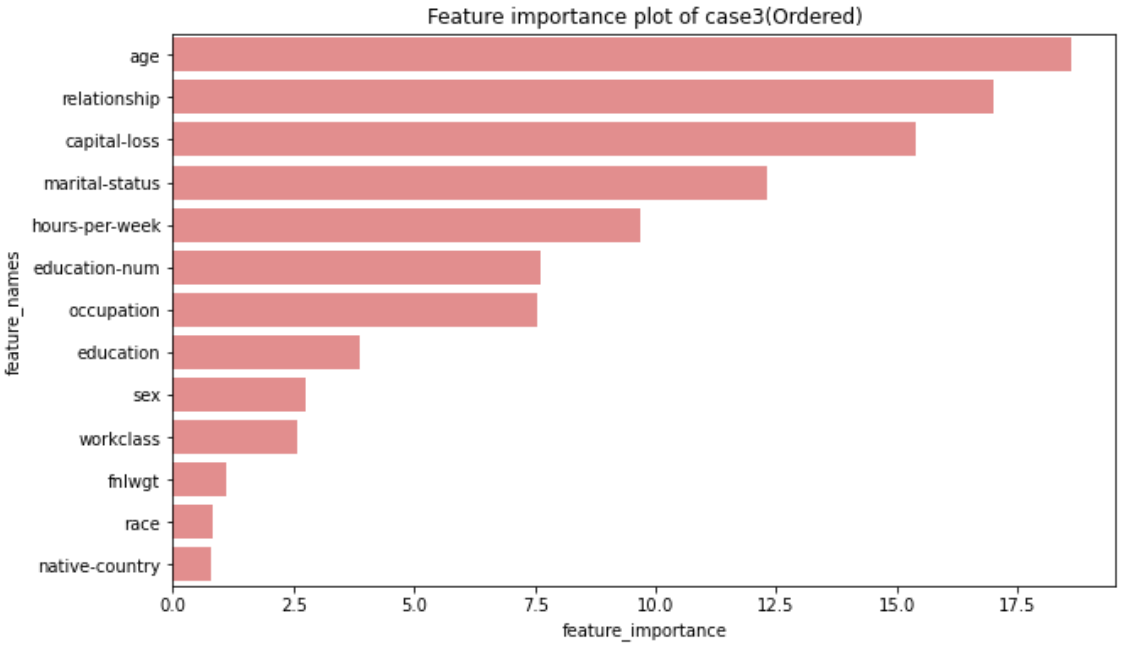


그림 25 Python으로 ordered Boosting 이면서 case3 일 때 학습한 모델에서의 feature importance plot

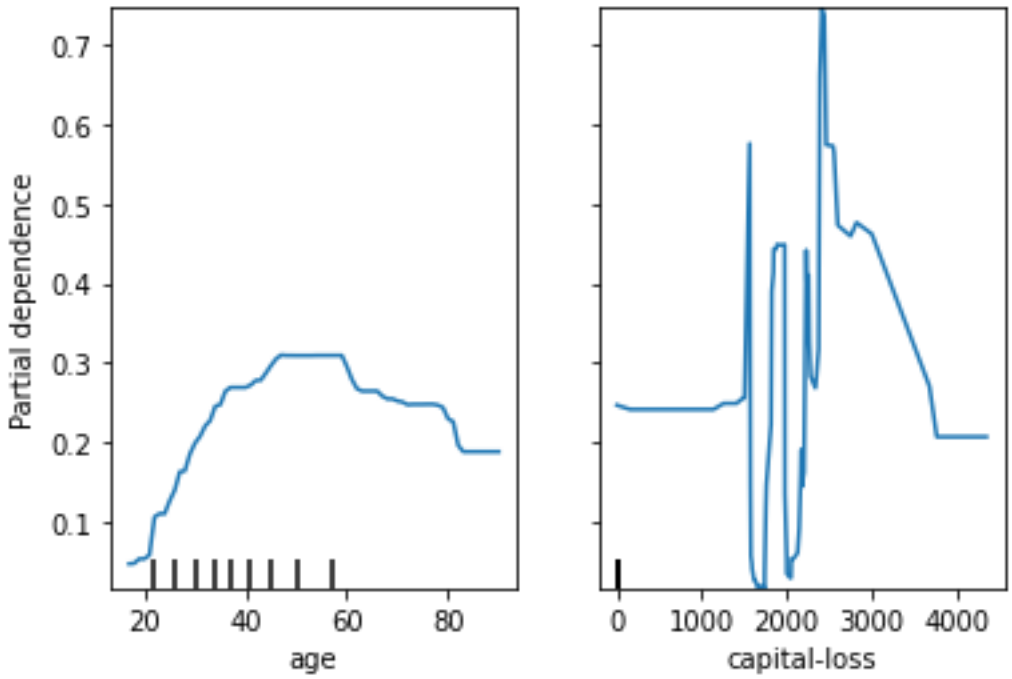


그림 26 case 1 ordered partial dependence plot

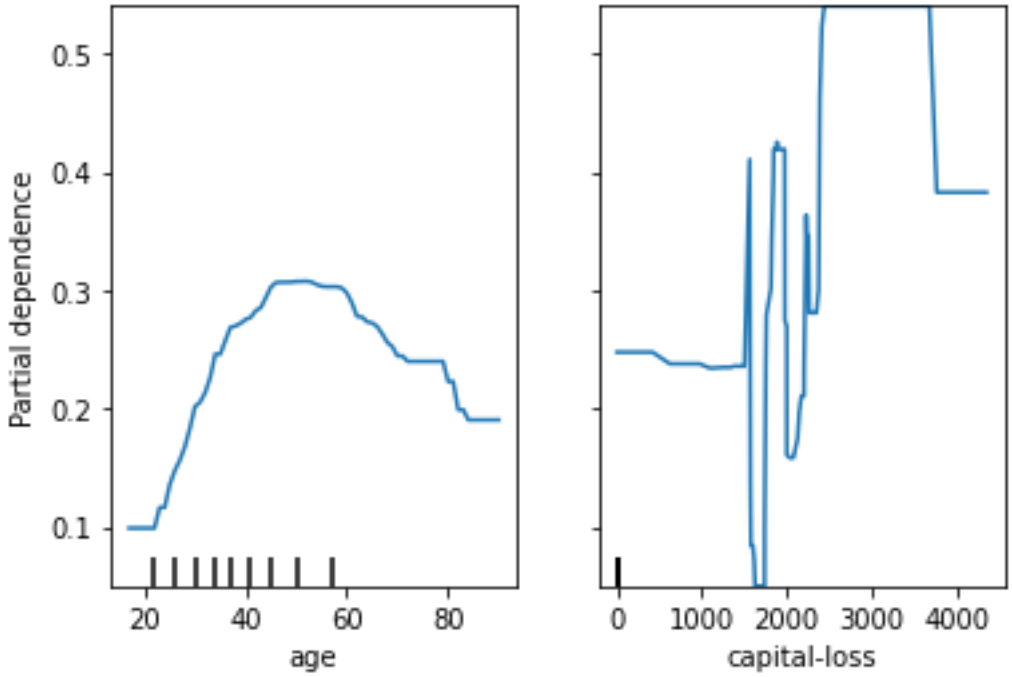


그림 27 case 1 plained partial dependence plot

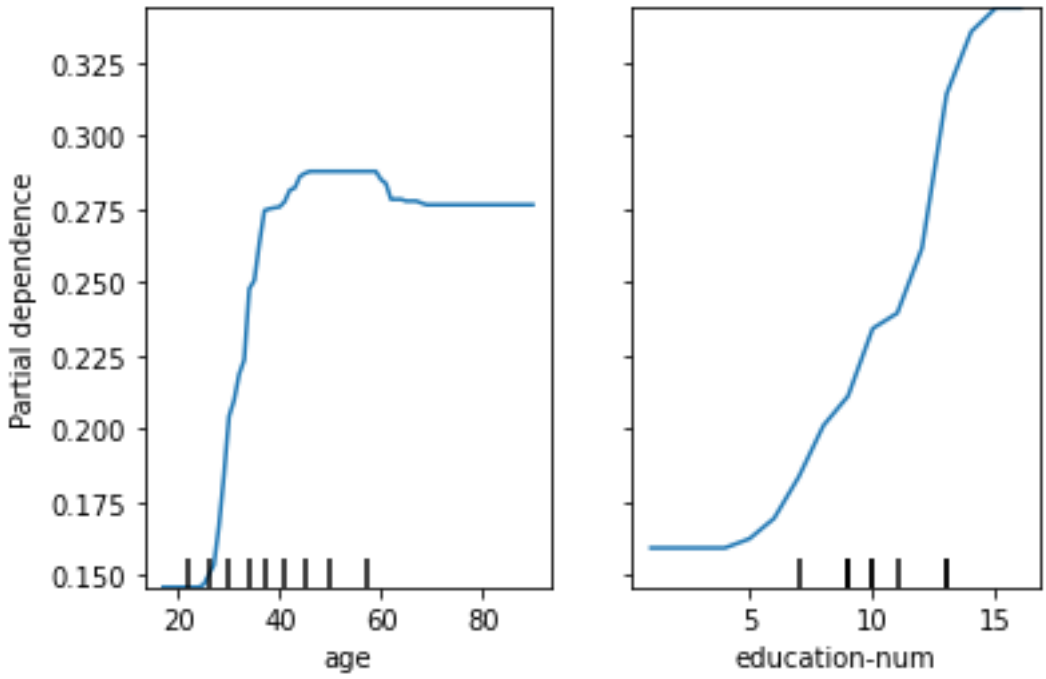


그림 28 case2 ordered partial dependence plot

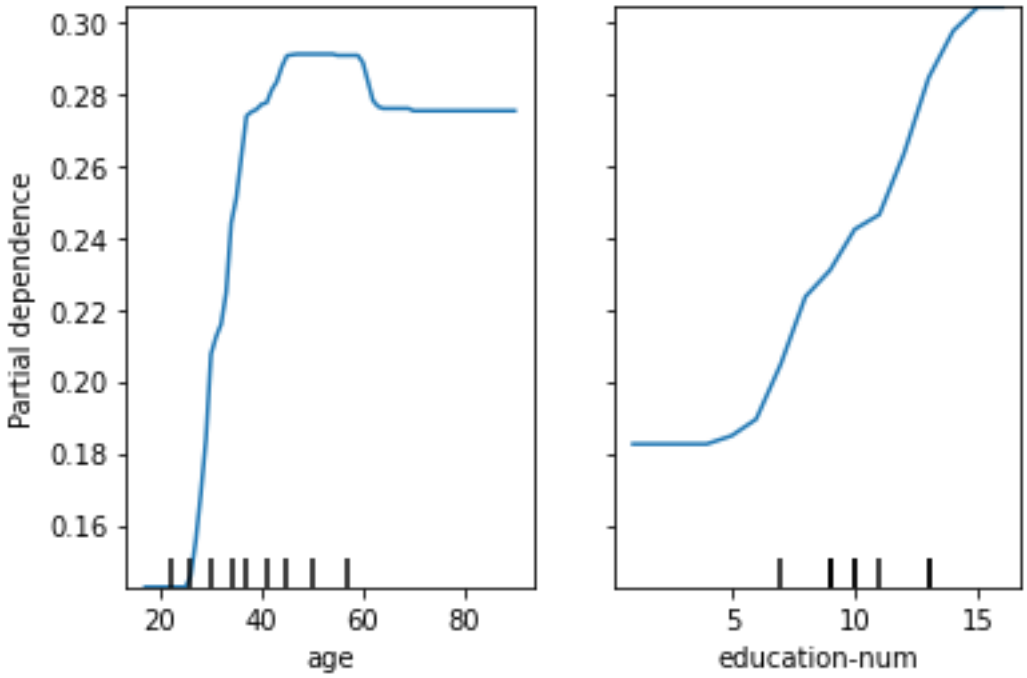


그림 29 case2 plain partial dependence plot

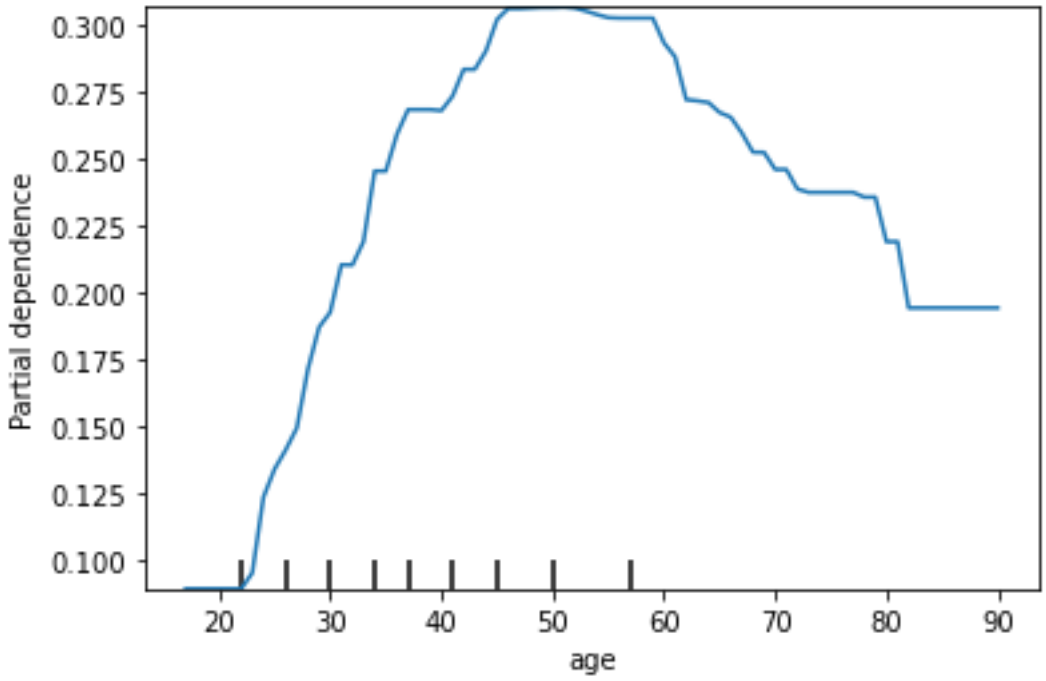


그림 30 case3 plain partial dependence plot

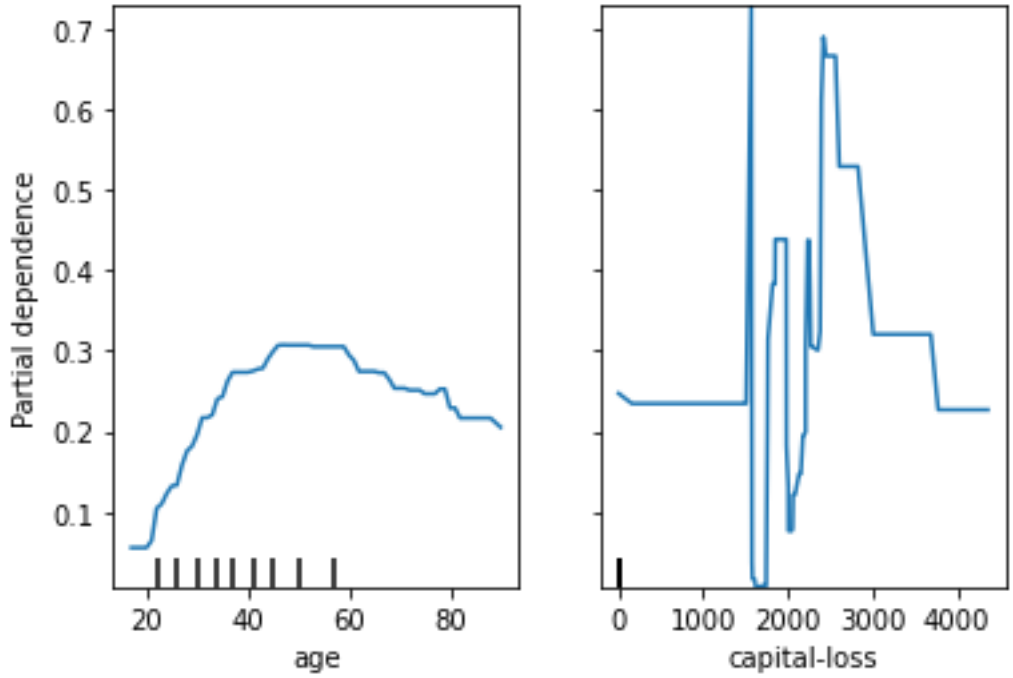


그림 31 case3 ordered partial dependence plot

XGBoost vs CatBoost

XGBoost와 CatBoost를 통한 예측 결과를 비교해본다. 밑의 표는 XGBoost와 CatBoost를 통한 예측에 대한 accuracy 값이다. 세 케이스 모두 각 모델의 성능에 큰 차이가 없다. Case1의 경우 catboost가 함수 내에서 파라미터 튜닝이 이뤄지기 문에 xgboost보다 더 좋은 성능을 보인다. 하지만, case3에서는 모델간에 큰 차이가 나지 않는다.

case	Tool	XGBoost	CatBoost-Plain	Catboost-Ordered
Case1	Python	0.8465	0.85	0.85
	R	0.8465	0.8495	0.851
Case2	Python	0.8377	0.84	0.84
	R	0.8377	0.8414	0.8391
case3 grid search	Python	0.85	0.85	0.85
	R	0.8508	0.8449	0.8444

표 8 모델별 accuracy비교

Appendix: Code (R and Python)

R 코드

```
##############################################

##################xgboost##################

##############################################



library(mltools)

library(data.table)

require(xgboost)

require(methods)

library(Matrix)

library(caret)

library(dplyr)

library(Metrics)



##############################################

#######if use train and test data set########

##############################################



train<-read.csv("C:/Users/heaju/Desktop/train_adult_onehot.csv",  header = TRUE, na.strings=c("-999", "NA", "","  ?"))

test<-read.csv("C:/Users/heaju/Desktop/test_adult_onehot.csv",  header = TRUE, na.strings=c("-999", "NA", "","  ?"))
```

```
#train$target <- as.factor(train$target)

#levels(train$target) <- c("first_class", "second_class")

head(train)


##############################################

##################modeling###################

##############################################



#predictor

train1 <- train %>% select(-(capital.gain))%>% select(-(target))

test1 <- test %>% select(-(capital.gain))%>% select(-(target))

train_mat <- as.matrix(train1)

test_mat <- as.matrix(test1)


#label

train.label <- unlist(train %>% select(target))

test.label <- unlist(test %>% select(target))

label_train <- as.numeric(train.label)

label_test <- as.numeric(test.label)


#prepare data set for xgboost

dtrain <- xgb.DMatrix(data = train_mat, label=label_train)

dtest <- xgb.DMatrix(data = test_mat)


##############################################
```

```
##################case1#####################

#############################################


set.seed(777)

param <- list(objective   = "binary:logistic",

              eval_metric = "error")


start_time <- Sys.time()

xgb <- xgb.train(params  = param,

                 data    = dtrain,

                 nround = 100)

end_time <- Sys.time()

end_time - start_time




XGB_pred <- predict(xgb, dtest)

XGB_pred <- ifelse(XGB_pred > 0.5, 1, 0)



# classification report

XGB_pred <- as.factor(XGB_pred)

label_test <- as.factor(label_test)

confusionMatrix(XGB_pred, label_test, mode = "everything", positive="1")
```

```
# variance importance

plot1 <- xgb.importance(colnames(dtrain), model = xgb) %>% xgb.plot.importance(top_n = 10)





data.frame(plot1) %>%

  ggplot(aes(reorder(Feature, Frequency), Frequency))+

  geom_bar(stat="identity", fill="lightcoral")+

  coord_flip()+theme_light()+

  labs(x="Feature", title="Feature importance for case1")+

  theme(plot.title = element_text(hjust=0.5))


# partial dependence plot

# train =  target 제외한 train matrix

partial(object = xgb, pred.var = "age", plot = TRUE, train = train_mat, rug = TRUE, plot.engine = "ggplot2")

partial(object = xgb, pred.var = "capital.gain", plot = TRUE, train = train_mat, rug = TRUE, plot.engine = "ggplot2")


##############################################

#############case2 grid search###############

##############################################



# set up the cross-validated hyper-parameter search

xgb_grid_1 = expand.grid(max_depth = c(2,4,6),

                         eta = c(0.05,0.1),
```

```
                colsample_bytree = c(0.6, 0.8),
                gamma = c(0.1, 0.5, 1),
                nrounds = c(500,1000),
                subsample = 1,
                min_child_weight = c(0.5,1))
# pack the training control parameters
xgb_trcontrol_1 = trainControl(method = "cv",
                    number = 5,
                    verboseIter = TRUE,
                    returnData = FALSE,
                    returnResamp = "all",                    # save losses across all models
                    classProbs = TRUE,                       # set to TRUE for AUC to be computed
                    summaryFunction = twoClassSummary,
                    allowParallel = TRUE)


# train the model for each parameter combination in the grid,
#   using CV to evaluate
xgb_train_1 = train(x = train_mat,
            y = as.factor(train$target),
            trControl = xgb_trcontrol_1,
            tuneGrid = xgb_grid_1,
            method = "xgbTree")



xgb_model$bestTune
```

```
##############################################

##################case2#####################

##############################################


set.seed(777)

param2 <- list(objective   = "binary:logistic",

               eval_metric = "error",

               eta         = 0.05,

               max_depth = 6,

               colsample_bytree = 0.6,

               gamma = 0.5,

               min_child_weight=0.5,

               subsample=1,

               lambda=1,

               alpha=0)


start_time <- Sys.time()

xgb2 <- xgb.train(params  = param2,

                  data    = dtrain,

                  nrounds = 500)

end_time <- Sys.time()

end_time - start_time


XGB_pred2 <- predict(xgb2, dtest)

XGB_pred2 <- ifelse(XGB_pred2 > 0.5, 1, 0)
```

```
# classification report

XGB_pred2 <- as.factor(XGB_pred2)

label_test <- as.factor(label_test)

confusionMatrix(XGB_pred2, label_test, mode = "everything", positive="1")



# variance importance

plot1 <- xgb.importance(colnames(dtrain), model = xgb2) %>% xgb.plot.importance(top_n = 10)





data.frame(plot1) %>%

  ggplot(aes(reorder(Feature, Frequency), Frequency))+

  geom_bar(stat="identity", fill="lightcoral")+

  coord_flip()+theme_light()+

  labs(x="Feature", title="Feature importance for case3")+

  theme(plot.title = element_text(hjust=0.5))




##############################################

##################catboost###################

##############################################
```

```
library(tidyverse)

library(devtools)

library(catboost)

library(caret)

#install.packages('devtools')

#devtools::install_url('https://github.com/catboost/catboost/releases/download/v0.20/catboost-R-Windows-0.20.tgz', INSTALL_opts = c("--no-multiarch", "--no-test-load"))



#install.packages("ggiraphExtra")

library(ggiraphExtra)

library(tibble)


##############################################

#######if use train and test data set########

##############################################



train<-read.csv("C:/Users/heaju/Desktop/train_adult_noonehot.csv",  header = TRUE, na.strings=c("-999", "NA", "","  ?"))

test<-read.csv("C:/Users/heaju/Desktop/test_adult_noonehot.csv",  header = TRUE, na.strings=c("-999", "NA", "","  ?"))

train <- train[,-1]

test <- test[,-1]



test$occupation <- as.factor(test$occupation)

test$education <- as.factor(test$education)
```

```
test$workclass <- as.factor(test$workclass)

test$marital.status <- as.factor(test$marital.status)

test$relationship <- as.factor(test$relationship)

test$race <- as.factor(test$race)

test$sex <- as.factor(test$sex)

test$native.country <- as.factor(test$native.country)

train$occupation <- as.factor(train$occupation)

train$education <- as.factor(train$education)

train$workclass <- as.factor(train$workclass)

train$marital.status <- as.factor(train$marital.status)

train$relationship <- as.factor(train$relationship)

train$race <- as.factor(train$race)

train$sex <- as.factor(train$sex)

train$native.country <- as.factor(train$native.country)



#ggplot(data = train,aes(x=reorder(education,euducation.num),y=euducation.num))+geom_boxplot()+labs(y="education.num",
x="education", subtitle="education vs education.num")

#############################################

###################case1#####################

#############################################



train_pool <- catboost.load_pool(

  data = train[, -15],

  label = train[, "target"],

  cat_features = which(sapply(train[, -15], is.factor))
```

```
)

test_pool <- catboost.load_pool(
  data = test[, -15],
  label = test[, "target"]
)


params <- list(
  boosting_type = "Plain",
  loss_function = 'Logloss',
  depth = 2,
  iterations=1000,
  learning_rate = 0.01,
  random_seed = 777
)

start <- Sys.time()
catboost1 <- catboost.train(learn_pool=train_pool, test_pool=test_pool, params=params)
catboost1_Runtime <- Sys.time() - start
catboost1_Runtime


predy1 <- catboost.predict(catboost1, test_pool)
predy1[predy1<0] = 0
predy1[predy1>0] = 1
```

```
confusionMatrix(as.factor(predy1),
            as.factor(test$target), mode = "everything", positive="1")



imp1 = catboost.get_feature_importance(catboost1,
                              pool = NULL,
                              type = 'FeatureImportance',
                              thread_count = -1)



impdf <- rownames_to_column(data.frame(imp1))
impdf %>% ggplot(aes(reorder(rowname, imp1), imp1))+
 geom_bar(stat='identity', fill="lightcoral")+
 coord_flip()+theme_light()+
 labs(x='feature', y='importance', title="Feature importance plot for case1(ordered)")+
 theme_light()+theme(plot.title = element_text(hjust=0.5))+
 theme(axis.text.x = element_text(angle = 30, vjust=0.5))



##############################################
#############case2 grid search###############
##############################################
```

```
start <- Sys.time()
fit_control <- trainControl(method = "cv",
                    number = 3,
                    classProbs = TRUE)

grid <- expand.grid(depth = c(2, 4, 6),
              learning_rate = c(0.05, 0.1),
              iterations = c(500, 1000),
              l2_leaf_reg = c(0.1, 0.5, 1),
              rsm = c(0.5, 0.85),
              border_count = c(64, 254))

report <- train(train[, -7], as.factor(make.names(train[, "target"])),
            method = catboost.caret,
            logging_level = 'Silent',
            preProc = NULL,
            tuneGrid = grid, trControl = fit_control,
            boosting_type = "Plain",
            metric = "Accuracy")
grid_Runtime <- Sys.time() - start
grid_Runtime
print(report)
```

```
##############################################

##################case2#####################

##############################################


params <- list(

  boosting_type = "Ordered",

  loss_function = 'Logloss',

  depth = 2,

  learning_rate=0.05,

  iterations=500,

  l2_leaf_reg=0.1,

  rsm=0.5,

  border_count=64,

  random_seed = 777

)

start <- Sys.time()

catboost2 <- catboost.train(learn_pool=train_pool, test_pool=test_pool, params=params)

catboost2_Runtime <- Sys.time() - start

catboost2_Runtime



predy2 <- catboost.predict(catboost2, test_pool)

predy2[predy2<0] = 0

predy2[predy2>0] = 1


confusionMatrix(as.factor(predy2),
```

```
        as.factor(test$target), mode = "everything", positive="1")
```

```
imp1 = catboost.get_feature_importance(catboost2,
                                       pool = NULL,
                                       type = 'FeatureImportance',
                                       thread_count = -1)
```

```
impdf <- rownames_to_column(data.frame(imp1))
impdf %>% ggplot(aes(reorder(rowname, imp1), imp1))+
 geom_bar(stat='identity', fill="lightcoral")+
 coord_flip()+theme_light()+
 labs(x='feature', y='importance', title="Feature importance plot for case2(ordered)")+
 theme_light()+theme(plot.title = element_text(hjust=0.5))+
 theme(axis.text.x = element_text(angle = 30, vjust=0.5))
```

Python code

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

import random

import time

import warnings

warnings.filterwarnings('ignore')


from sklearn.model_selection import train_test_split, GridSearchCV


import xgboost

from xgboost import plot_importance


from sklearn.metrics import mean_squared_error

import xgboost as xgb

from xgboost import XGBClassifier

train = pd.read_csv('train_adult_onehot.csv', na_values='?')

test = pd.read_csv('test_adult_onehot.csv', na_values='?')



train.head()


test.head()
```

```
test.columns

train_x = train.drop(['capital-gain','target'], axis=1)
train_y = train['target']

test_x = test.drop(['capital-gain','target'], axis=1)
test_y = test['target']
train_x.head()

train_x.columns

# case1

start = time.time()
xgb = XGBClassifier(random_state=777)
xgb.fit(train_x, train_y)
print("실행시간:{}".format(time.time()-start))

xgb.set_params

pred_y = xgb.predict(test_x)
pred_y

from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

```
con_mat = confusion_matrix(pred_y,test_y)

con_mat



report = classification_report(test_y, pred_y)

print(report)


fig, ax = plt.subplots(1, 1, figsize=(10, 7))

plot_importance(xgb, ax=ax, color="lightcoral", max_num_features=10)

plt.title("Feature importance for case 1");


con_mat = confusion_matrix(pred_y,test_y)

con_mat


from sklearn.inspection import plot_partial_dependence

plot_partial_dependence(xgb, features=['age','education-num','hours-per-week'],X=train_x)


# 별로

plot_partial_dependence(xgb, features=[('age','capital-gain')],X=train_x)

plt.tight_layout()


# case2


start = time.time()

xgb = XGBClassifier(n_estimators = 1000, learning_rate=0.01,max_depth=2, random_state=777)

xgb.fit(train_x, train_y)
```

```
print("실행시간:{}".format(time.time()-start))


xgb.set_params


pred_y = xgb.predict(test_x)

pred_y


from sklearn.metrics import confusion_matrix

from sklearn.metrics import classification_report


con_mat = confusion_matrix(pred_y,test_y)

con_mat



report = classification_report(test_y, pred_y)

print(report)


fig, ax = plt.subplots(1, 1, figsize=(10, 7))

plot_importance(xgb, ax=ax, color="lightcoral", max_num_features=10)

plt.title("Feature importance for case 2");


con_mat


from sklearn.inspection import plot_partial_dependence

plot_partial_dependence(xgb, features=['age','education-num','capital-loss'],X=train_x)
```

```
# case3

start = time.time()

xgb2 = XGBClassifier()

xgb_grid = {

    'max_depth':[2,4,6],

    'colsample_bytree':[0.6, 0.8],

    'gamma': [0.1, 0.5, 1],

    'min_child_weight':[0.5,1],

    'n_estimators':[500, 1000],

    'learning_rate':[0.05, 0.1],

    'random_state':[777]

}

xgbCV = GridSearchCV(xgb2, xgb_grid, verbose=2)

xgbCV.fit(train_x, train_y)

print("실행시간:{}".format(time.time()-start))

print(xgbCV.best_params_)


from sklearn.metrics import classification_report

from sklearn.metrics import confusion_matrix


start = time.time()

xgb = XGBClassifier(n_estimators = 500, learning_rate=0.05,max_depth=4,gamma=0.1,colsample_bytree=0.8,min_child_weight=0.5,
random_state=777)

xgb.fit(train_x, train_y)

print("실행시간:{}".format(time.time()-start))


pred_y = xgb.predict(test_x)
```

```
pred_y

report = classification_report(test_y, pred_y)
print(report)

con_mat = confusion_matrix(pred_y,test_y)
con_mat

fig, ax = plt.subplots(1, 1, figsize=(10, 7))
plot_importance(xgb, ax=ax, color="lightcoral", max_num_features=18)
plt.title("Feature importance for case 3");

con_mat

from sklearn.inspection import plot_partial_dependence
plot_partial_dependence(xgb, features=['fnlwgt','age','hours-per-week'],X=train_x)

## Catboost

pip install catboost

from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from catboost import CatBoostClassifier, Pool, metrics, cv
from sklearn.metrics import accuracy_score
```

```
from sklearn.model_selection import KFold

from sklearn.linear_model import RidgeCV

import matplotlib.pyplot as plt

from functools import partial

import seaborn as sns

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import random

import time

import warnings

from catboost import CatBoostClassifier


warnings.filterwarnings('ignore')


train = pd.read_csv('train_adult_noonehot.csv')

test = pd.read_csv('test_adult_noonehot.csv')


train.head()


train["workclass"] = train["workclass"].astype('category')

train["education"] = train["education"].astype('category')

train["marital-status"] = train["marital-status"].astype('category')

train["occupation"] = train["occupation"].astype('category')

train["relationship"] = train["relationship"].astype('category')

train["sex"] = train["sex"].astype('category')
```

```
train["race"]=train["race"].astype('category')

train["native-country"] = train["native-country"].astype('category')



test["workclass"] = test["workclass"].astype('category')

test["education"] = test["education"].astype('category')

test["marital-status"] = test["marital-status"].astype('category')

test["occupation"] = test["occupation"].astype('category')

test["relationship"] = test["relationship"].astype('category')

test["sex"] = test["sex"].astype('category')

test["race"]=test["race"].astype('category')

test["native-country"] = test["native-country"].astype('category')


train.dtypes


train.head(3)


train = train.drop('Unnamed: 0',axis=1)

test = test.drop('Unnamed: 0',axis=1)

train.head()


train_x = train.drop(['capital-gain','target'], axis=1)

train_y = train['target']

test_x = test.drop(['capital-gain','target'], axis=1)

test_y = test['target']

train_x.head()
```

```
test_x.head()

test_y.head()

train_x.columns

test_x.columns

train_x.dtypes

test_x.dtypes

cat_cols = train_x.columns[train_x.dtypes == 'category'].to_list()
num_cols = train_x.columns[train_x.dtypes != 'category'].to_list()

cat_cols

# case1



start = time.time()
clf = CatBoostClassifier(
    boosting_type="Plain",
    iterations=1000,
    random_state=777,
```

```
    loss_function='Logloss'
)



clf.fit(train_x, train_y,
        eval_set=(test_x, test_y),
        verbose=True,
        cat_features = cat_cols
)


print("실행시간:{}".format(time.time()-start))


print('CatBoost model is fitted: ' + str(clf.is_fitted()))
print('CatBoost model parameters:')
print(clf.get_params())


preds_class = clf.predict(test_x)
preds_class


from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report



print(classification_report(test_y, preds_class))
temp = pd.DataFrame({'feature_importance': clf.get_feature_importance(),
            'feature_names': train_x.columns}).sort_values(by=['feature_importance'],
```

```
                                ascending=False)


fig = plt.figure(figsize=(10, 6))

sns.barplot(x='feature_importance', y='feature_names', data=temp, color="lightcoral")

plt.title("Feature importance plot of case1(ordered)");


con_mat = confusion_matrix(preds_class,test_y)

con_mat


train_x.dtypes


from sklearn.inspection import plot_partial_dependence

plot_partial_dependence(clf, features=['age','capital-loss'],X=train_x)


pip install PDPbox

from pdpbox.pdp import pdp_isolate, pdp_plot

from pdpbox import pdp




pdp_season = pdp.pdp_isolate(

   model=model, dataset=X_train, model_features=X_train.columns,

   feature=['spring', 'summer', 'fall', 'winter']

)

fig, axes = pdp.pdp_plot(pdp_season,'season', center=False, cluster=True,n_cluster_centers=2,\

                 plot_lines=True, x_quantile=True, show_percentile=True)
```

```
##### case2 Ordered



start = time.time()

clf = CatBoostClassifier(

   boosting_type="Ordered",

   iterations=1000,

   random_state=777,

   depth = 2,

   learning_rate = 0.01,

   loss_function='Logloss'

)



clf.fit(train_x, train_y,

      eval_set=(test_x, test_y),

      verbose=True,

      cat_features = cat_cols

)


print("실행시간:{}".format(time.time()-start))


print('CatBoost model is fitted: ' + str(clf.is_fitted()))

print('CatBoost model parameters:')

print(clf.get_params())
```

```
preds_class = clf.predict(test_x)

preds_class


from sklearn.metrics import confusion_matrix

from sklearn.metrics import classification_report



print(classification_report(test_y, preds_class))

temp = pd.DataFrame({'feature_importance': clf.get_feature_importance(),

        'feature_names': train_x.columns}).sort_values(by=['feature_importance'],

                                    ascending=False)


fig = plt.figure(figsize=(10, 6))

sns.barplot(x='feature_importance', y='feature_names', data=temp, color="lightcoral")

plt.title("Feature importance plot of case2(ordered)");


con_mat = confusion_matrix(preds_class,test_y)

con_mat


from sklearn.inspection import plot_partial_dependence

plot_partial_dependence(clf, features=['age','education-num'],X=train_x)


####### case2 plain



start = time.time()
```

```
clf = CatBoostClassifier(
   boosting_type="Plain",
   iterations=1000,
   random_state=777,
   depth = 2,
   learning_rate = 0.01,
   loss_function='Logloss'
)



clf.fit(train_x, train_y,
      eval_set=(test_x, test_y),
      verbose=True,
      cat_features = cat_cols
)

print("실행시간:{}".format(time.time()-start))

print('CatBoost model is fitted: ' + str(clf.is_fitted()))
print('CatBoost model parameters:')
print(clf.get_params())

preds_class = clf.predict(test_x)
preds_class

from sklearn.metrics import confusion_matrix
```

```
from sklearn.metrics import classification_report



print(classification_report(test_y, preds_class))

temp = pd.DataFrame({'feature_importance': clf.get_feature_importance(),

            'feature_names': train_x.columns}).sort_values(by=['feature_importance'],

                                        ascending=False)


fig = plt.figure(figsize=(10, 6))

sns.barplot(x='feature_importance', y='feature_names', data=temp, color="lightcoral")

plt.title("Feature importance plot of case2(Plain)");


con_mat = confusion_matrix(preds_class,test_y)

con_mat


from sklearn.inspection import plot_partial_dependence

plot_partial_dependence(clf, features=['age','education-num'],X=train_x)


#### gird search


## grid search


from catboost import CatBoostRegressor


model = CatBoostClassifier(loss_function='Logloss',boosting_type="Ordered",random_seed=777,

                        cat_features = cat_cols) #or AUC
```

```
grid = {'learning_rate': [0.05,0.1],
        'depth': [2,4,6],
        'l2_leaf_reg': [0.1,0.5,1],
        'iterations':[500,1000],
        'border_count':[64,254],
        'rsm':[0.5,0.85]
        }

#cv default=3
start = time.time()
grid_search_result = model.grid_search(grid,
                                       X=train_x,
                                       y=train_y,
                                       plot=True)
print("실행시간:{}".format(time.time()-start))

grid_search_result
```

```
0.09*60
```

```
from catboost import CatBoostClassifier
```

```
model_clf = CatBoostClassifier(
   boosting_type="Ordered",
   border_count=64,
   l2_leaf_reg=1,
   learning_rate=0.1,
   rsm=0.85,
   depth=6,
   iterations=500,
   random_state=777,
   loss_function='Logloss'
)

start = time.time()
model_clf.fit(train_x, train_y,
      eval_set=(test_x, test_y),
      verbose=True,
      cat_features = cat_cols
)

print("실행시간:{}".format(time.time()-start))

print('CatBoost model is fitted: ' + str(model_clf.is_fitted()))
print('CatBoost model parameters:')
print(model_clf.get_params())

preds_class2 = model_clf.predict(test_x)
```

```
from sklearn.metrics import classification_report


print(classification_report(test_y, preds_class2))


temp = pd.DataFrame({'feature_importance': model_clf.get_feature_importance(),
            'feature_names': train_x.columns}).sort_values(by=['feature_importance'],
                                                ascending=False)


fig = plt.figure(figsize=(10, 6))
sns.barplot(x='feature_importance', y='feature_names', data=temp, color="lightcoral")
plt.title("Feature importance plot of case3(Ordered)");


con_mat = confusion_matrix(preds_class2,test_y)
con_mat



from sklearn.inspection import plot_partial_dependence
plot_partial_dependence(model_clf, features=['age', 'capital-loss'],X=train_x)


print (train['relationship'].cat.categories)
```